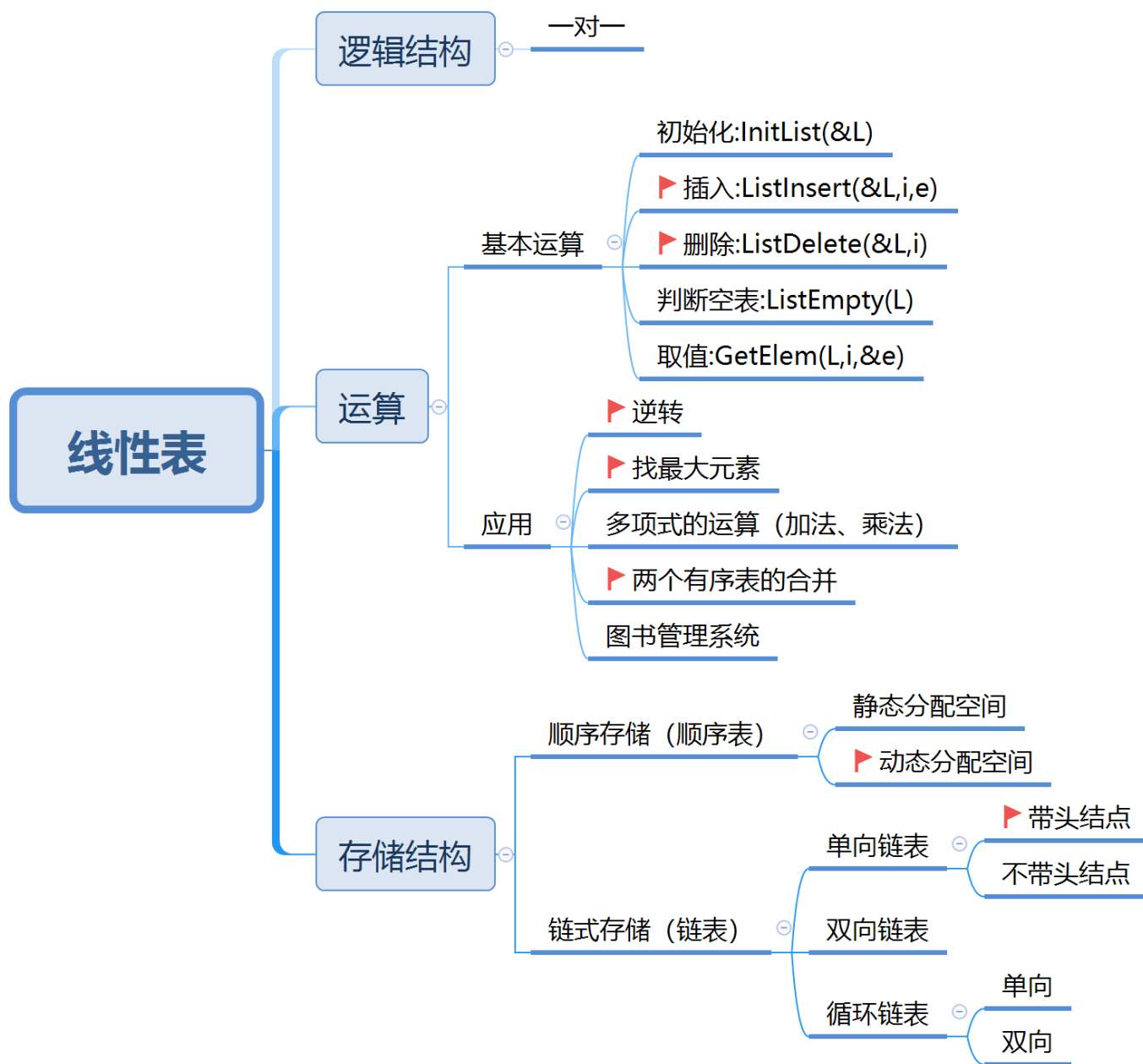




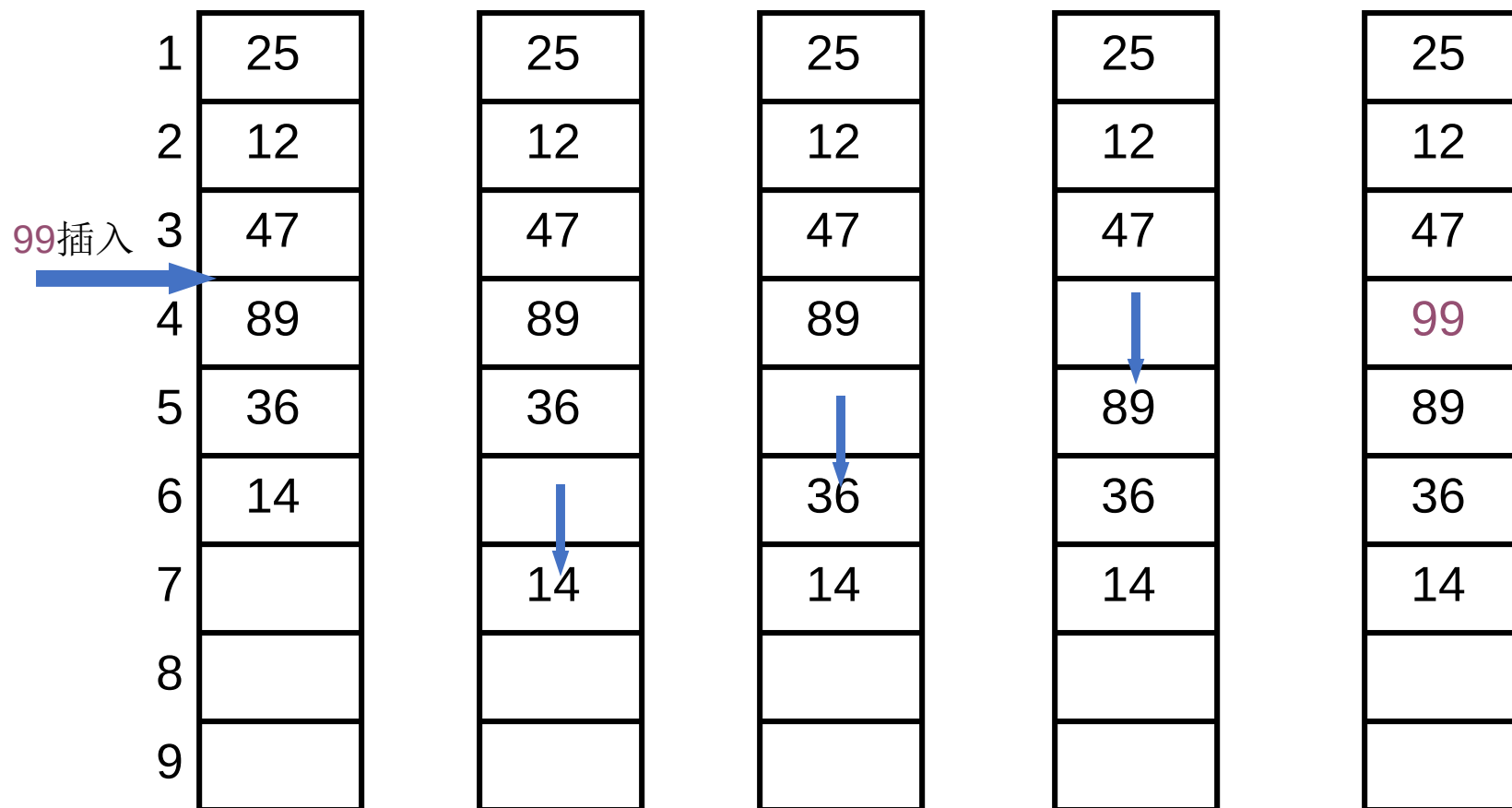
顺序表的表示

```
#define MAXSIZE 100
typedef struct {
    ElemType *elem;
    int length;
} SqList;
```

例： (34, 23, 67, 43)



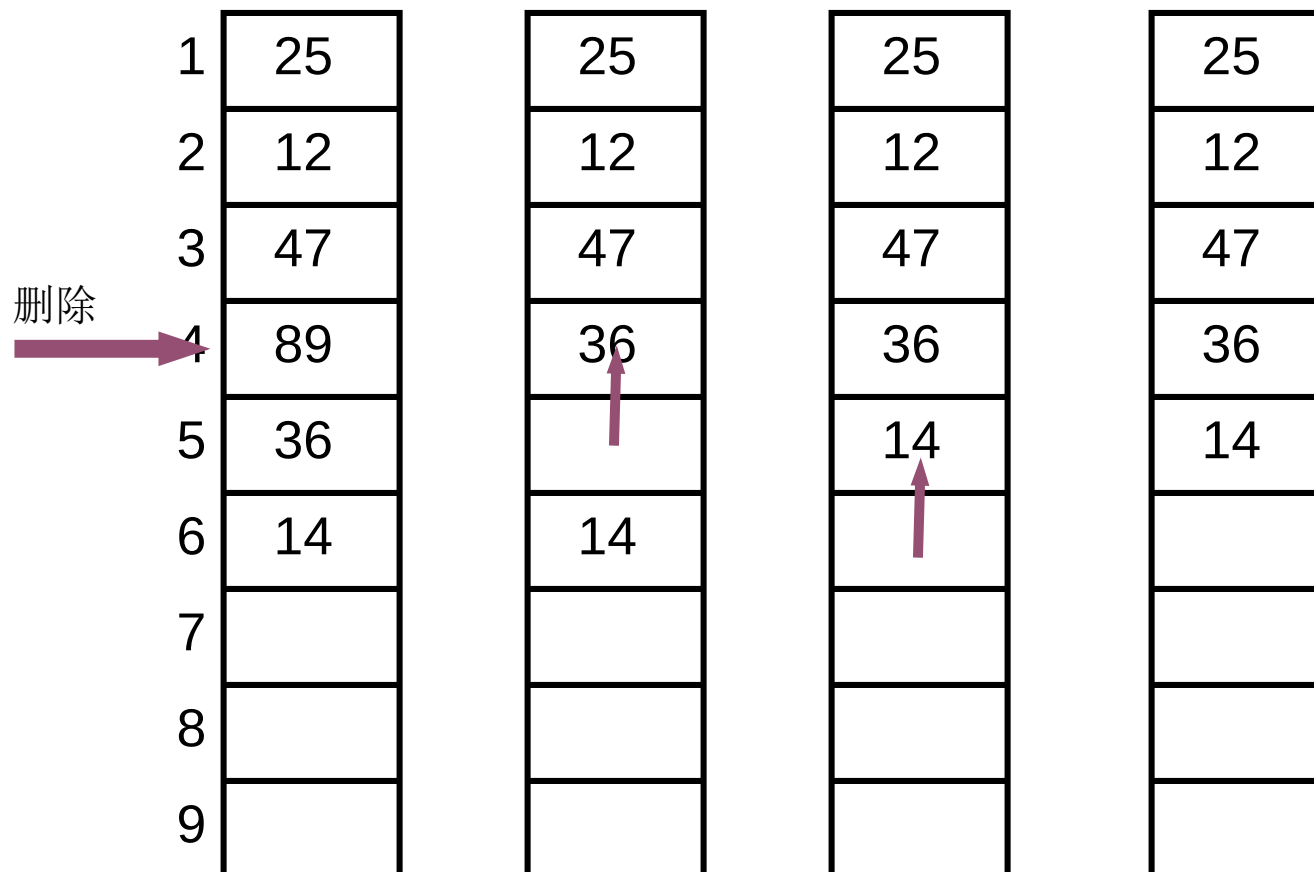
插入（插在第 i 个结点之前）



插第 4 个结点之前，移动 $6-4+1$ 次

插在第 i 个结点之前，移动 $n-i+1$ 次

删除（删除第 i 个结点）

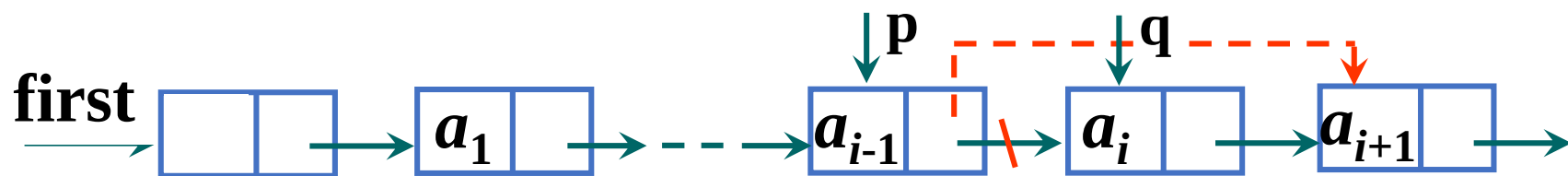


删除第 4 个结点，移动 $6-4$ 次

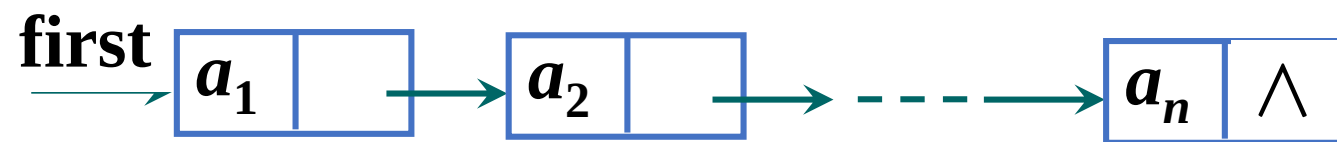
删除第 i 个结点，移动 $n-i$ 次

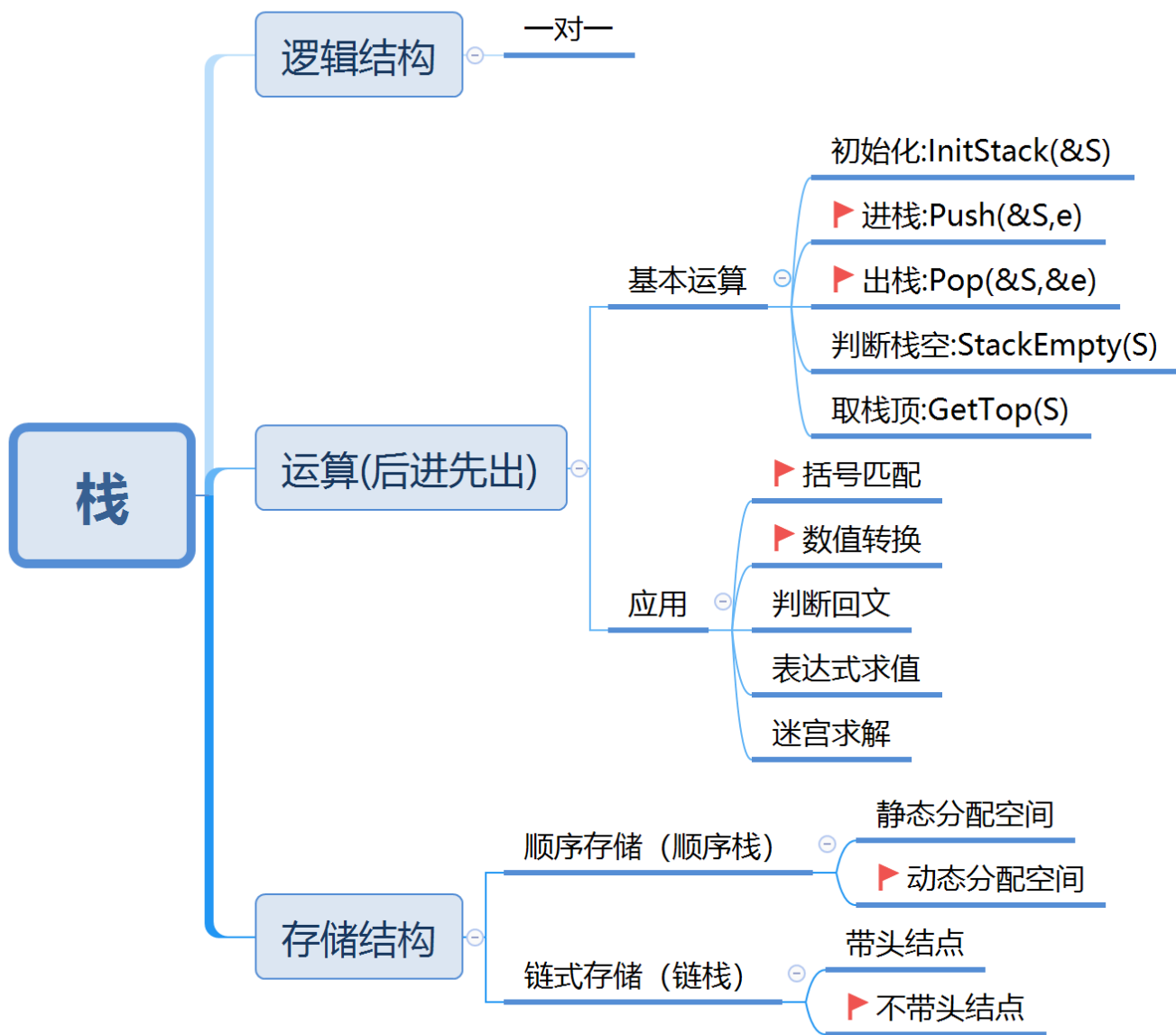
链表的表示

```
typedef struct Node{  
    ElemType    data;  
    struct Node *next  
} LNode, *LinkList;
```



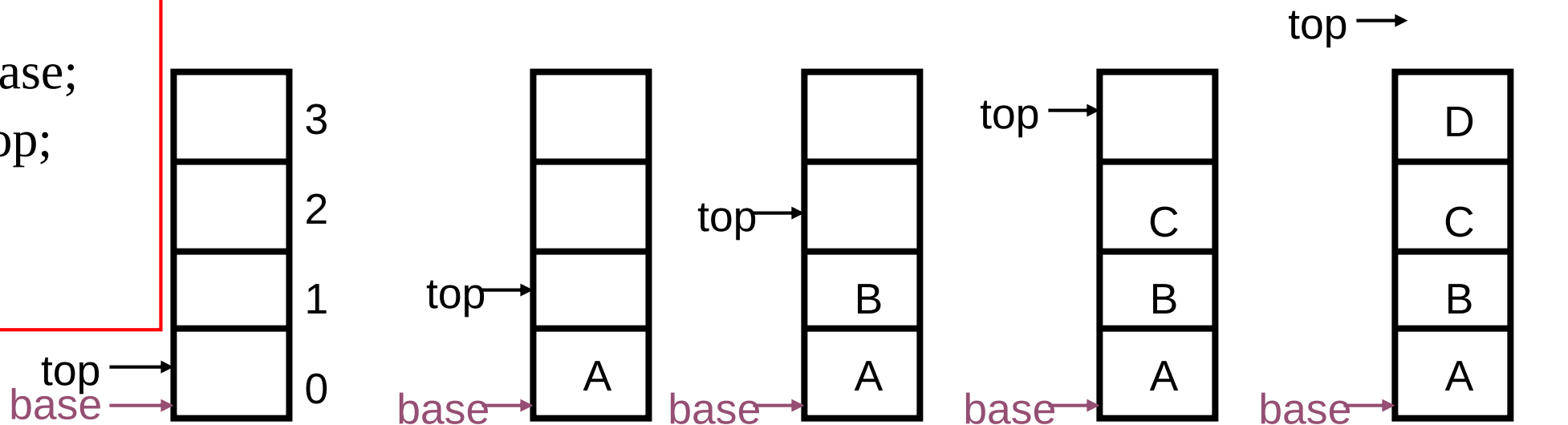
非空表





顺序栈的表示

```
#define MAXSIZE 100
typedef struct
{
    SElemType *base;
    SElemType *top;
    int stacksize;
} SqStack;
```



空栈

$base == top$ 是栈空标志

$stacksize = 4$

$top - base == stacksize$ 栈满

top 指真正的栈顶元素之上的下标地址

栈的应用 (1) : 数值转换

例如: $(1348)_{10} = (2504)_8$, 其运算过程如下:

	N	N div 8	N mod 8	
计算顺序 ↓	1348	168	4	↑ 输出顺序
	168	21	0	
	21	2	5	
	2	0	2	

栈的应用 (2) : 判断回文

问题:

abcdcba

返回true;

abcd

返回false。

栈的应用 (3) : 括号配对

问题:

[][()] () [] #

匹配成功, 返回**true**;

[) [] () #

匹配失败, 返回**false**。

栈的应用 (4) : 表达式求值

问题:

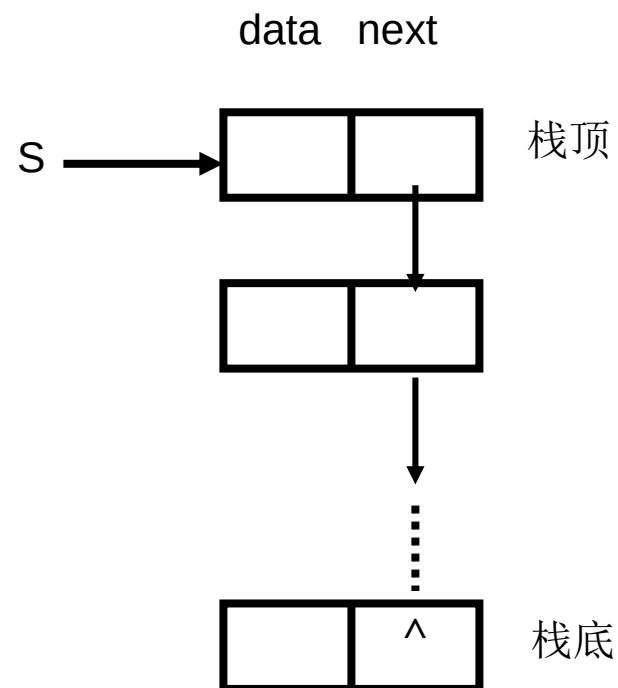
对表达式 $3 * (7 - 2)$ #求值, 结果为9

算术四则运算规则

- (1) 先乘除, 后加减
- (2) 从左算到右
- (3) 先括号内, 后括号外

链栈的表示

```
typedef struct StackNode {  
    SElemType data;  
    struct StackNode *next;  
} StackNode, *LinkStack;  
LinkStack S;
```



队列

逻辑结构

一对一

运算 (先进先出)

基本运算

初始化: `InitQueue(&Q)`

▶ 进队: `EnQueue(&Q,e)`

▶ 出队: `DeQueue(&Q,&e)`

判断队空: `QueueEmpty(Q)`

取队头元素: `GetHead(Q)`

应用

树的层次遍历

舞伴问题

图的BFS

迷宫求解

存储结构

▶ 顺序存储 (循环队列)

假溢出

循环利用空间

解决队空和队满条件矛盾问题

链式存储 (链队列)

单向链表

▶ 带头结点

最后一个元素出队列, 注意恢复队空的状态

不带头结点

循环链表

单向循环

设尾指针比设头指针好

双向循环

链队列

```
typedef struct QNode{
```

```
    QElemType  data;
```

```
    struct Qnode  *next;
```

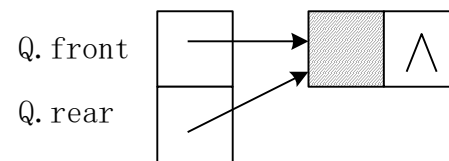
```
}Qnode, *QueuePtr;
```

```
typedef struct {
```

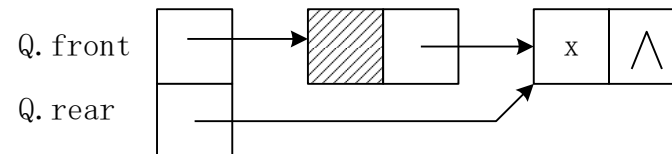
```
    QueuePtr  front;  //队头指针
```

```
    QueuePtr  rear;   //队尾指针
```

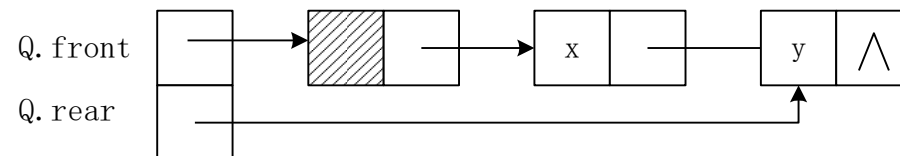
```
}LinkQueue;
```



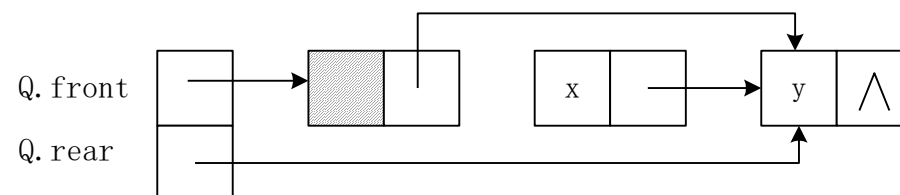
(a) 空队列



(b) 元素x入队列



(c) 元素y入队列



(d) 元素x出队列

队列的顺序表示

```
#define M 100
```

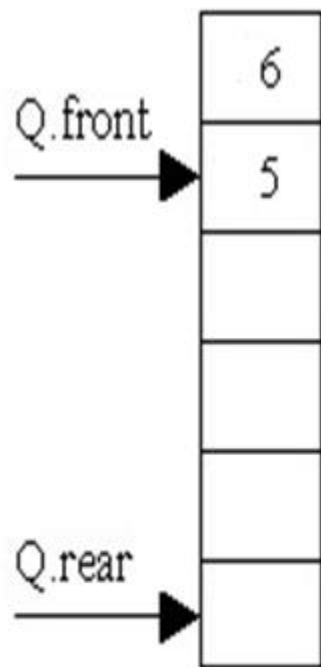
```
Typedef struct {
```

```
    QElemType *base;
```

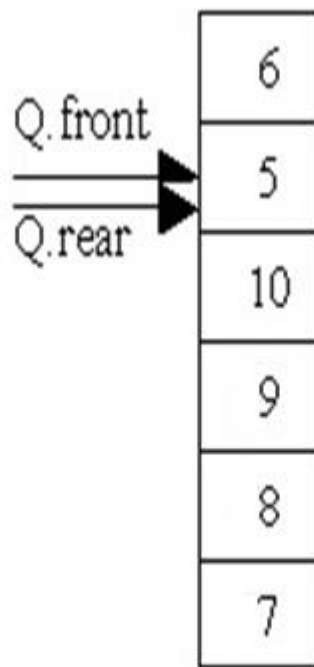
```
    int front; //队头下标
```

```
    int rear; //队尾下标
```

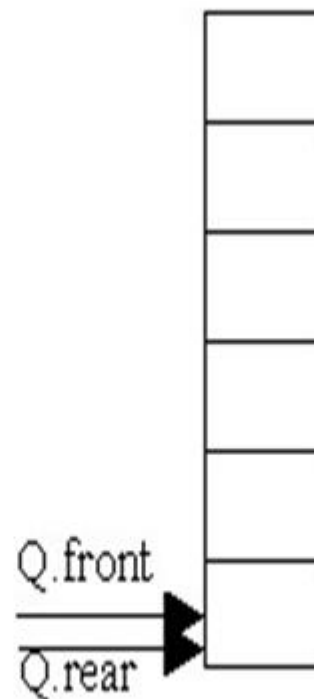
```
}SqQueue;
```



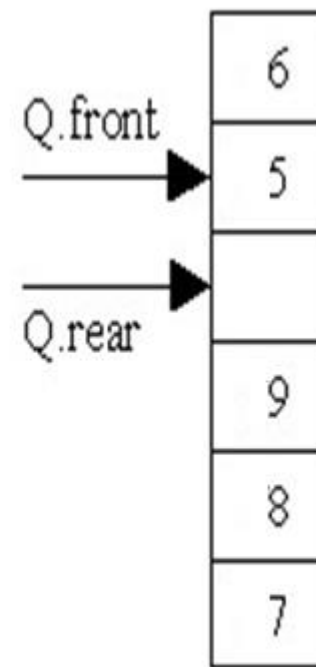
(a) 一般情况



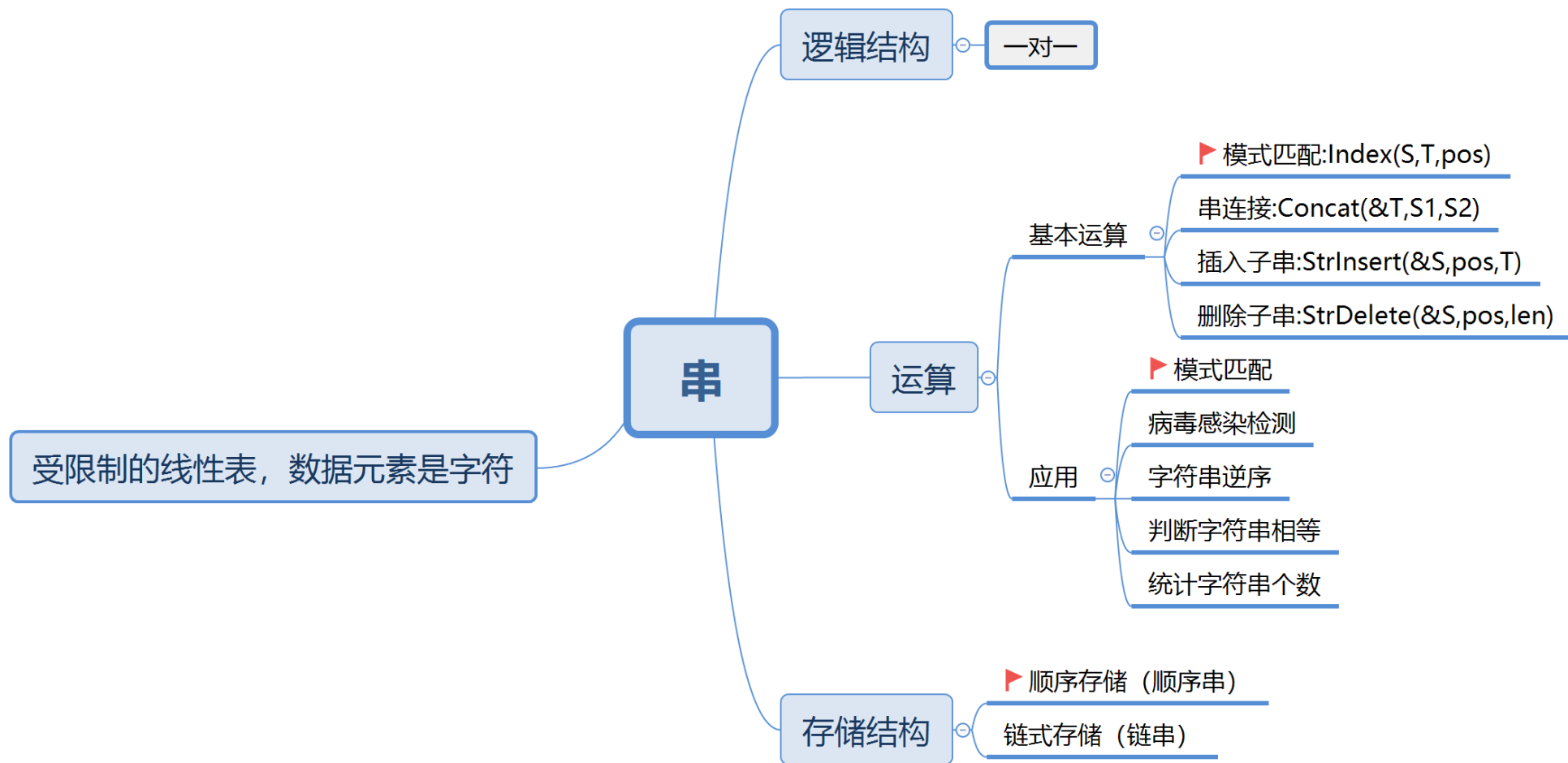
(b) 队列空间被占满



(c) 空队



(d) 呈“满”状态的循环队列



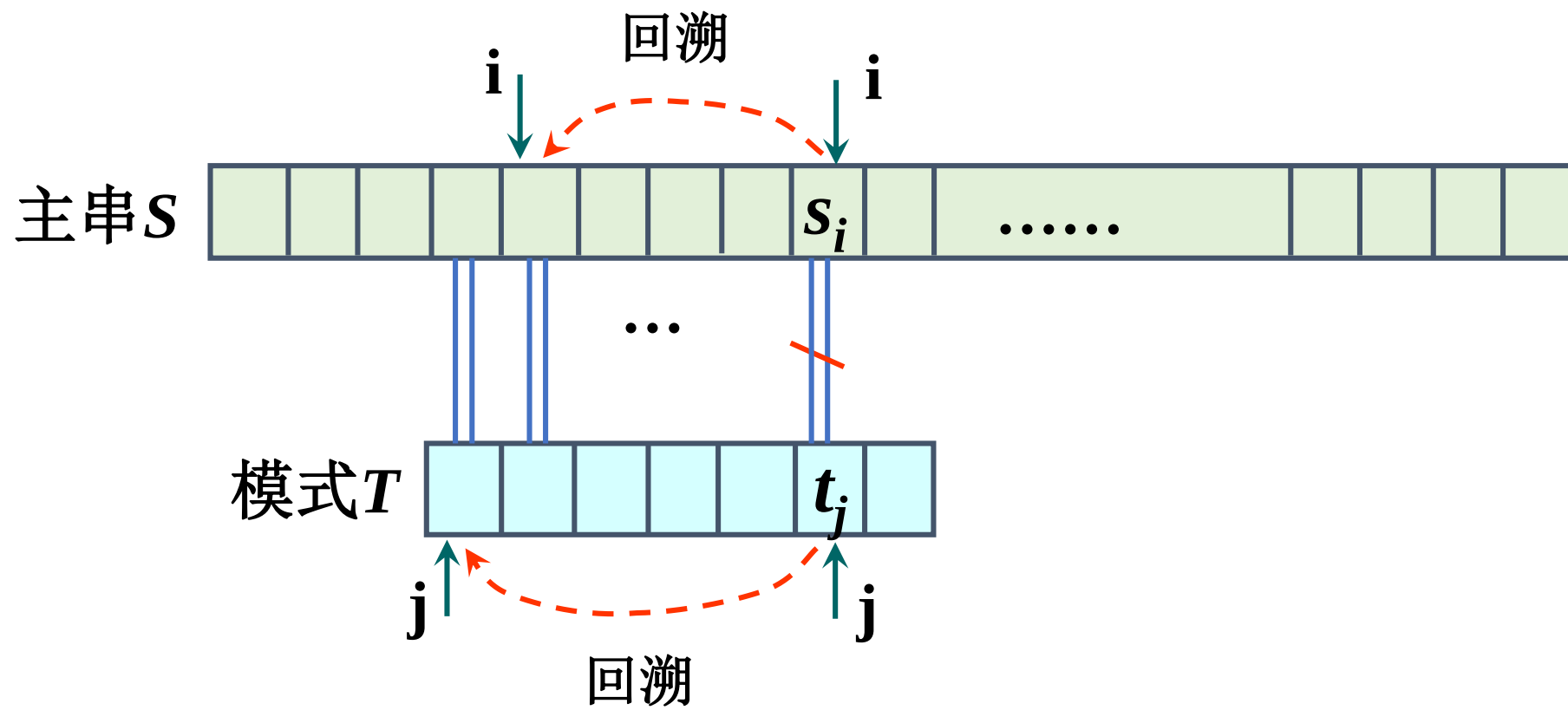
顺序串

```
typedef struct {  
    char *ch;  
    int  length;  
} HString;
```

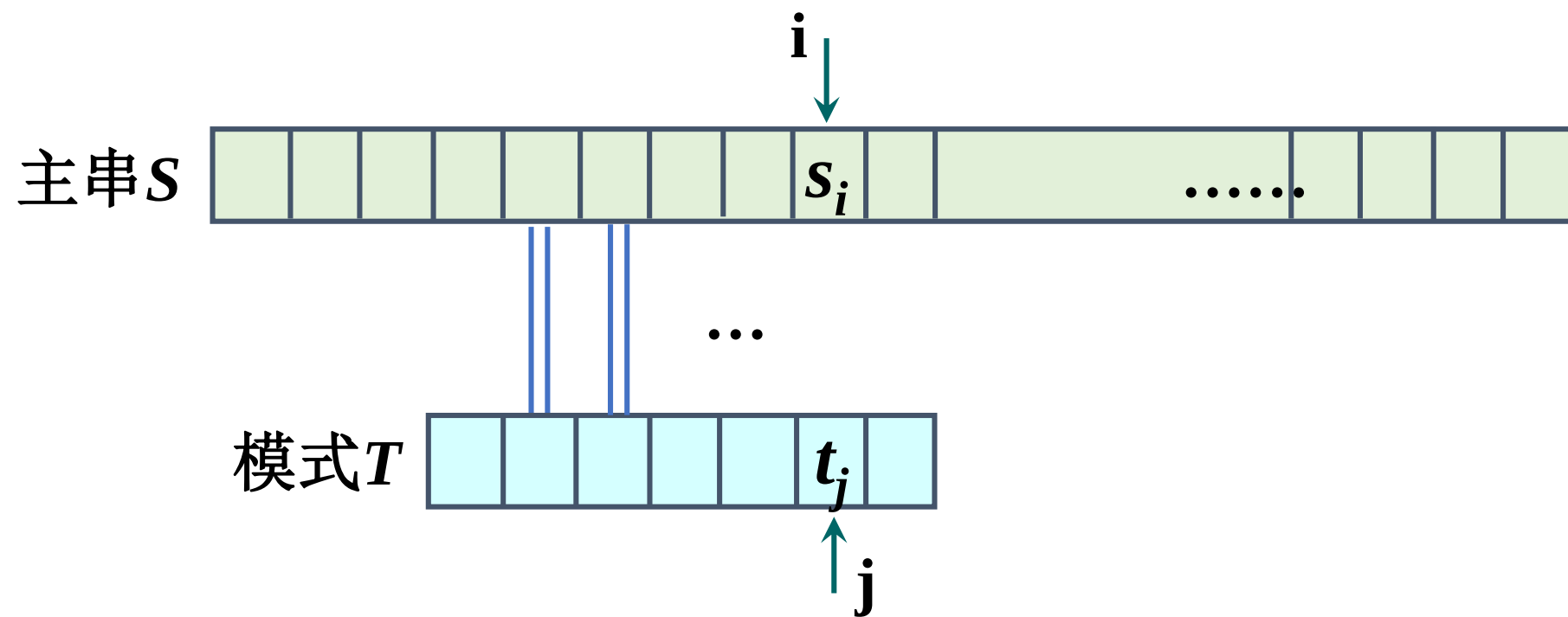
```
#define MAXSTRLEN 255    //用户可在255以内定义最长串长  
typedef char SString[MAXSTRLEN+1]; //0号单元存放串的长度
```

模式匹配——BF算法

例：主串 S ="ababc**abcac**bab"，模式 T ="ab**cac**"

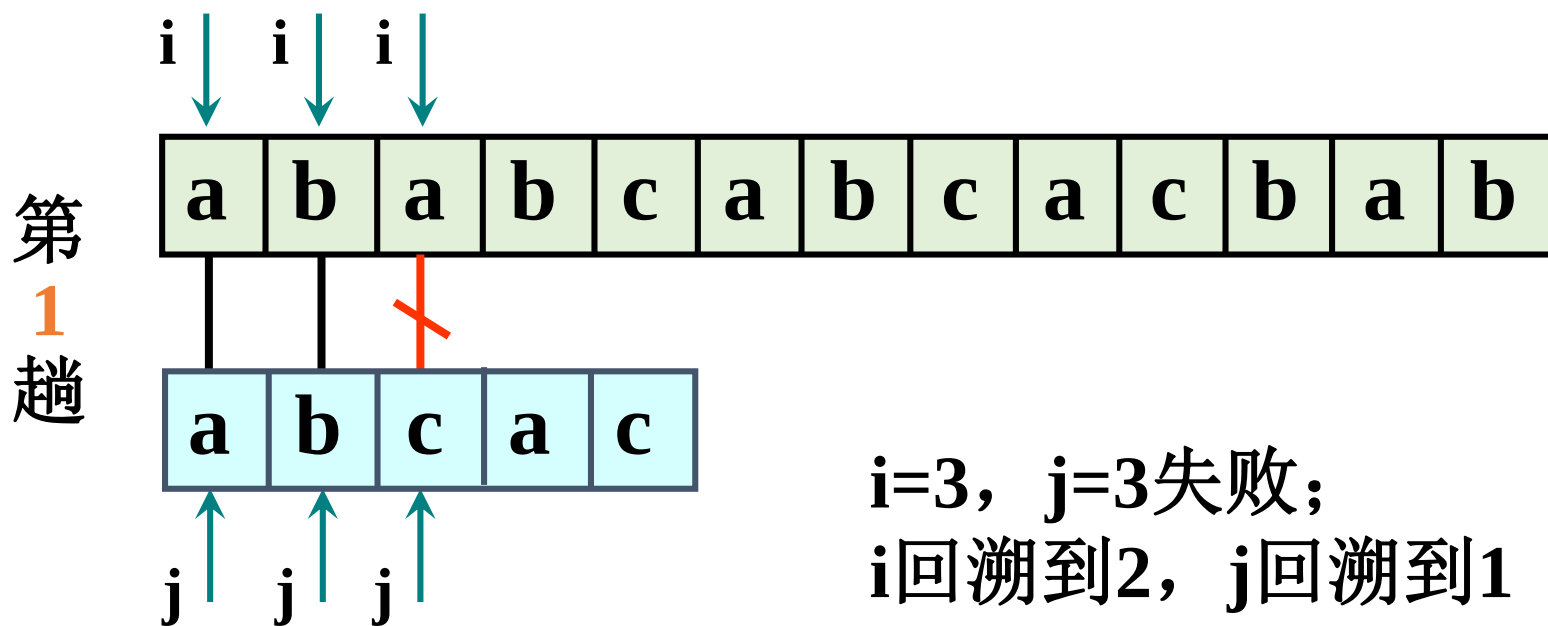


模式匹配——BF算法



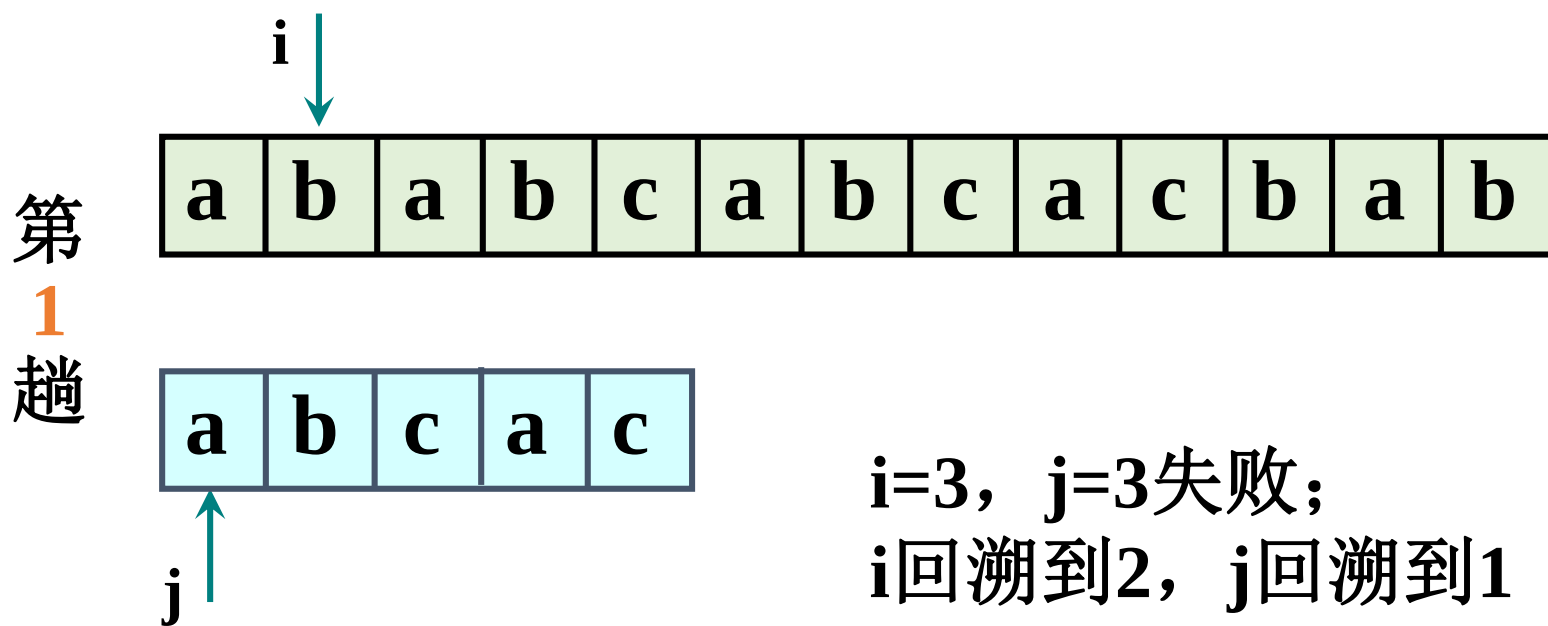
模式匹配——BF算法

例：主串S="ababc**abcac**bab"，模式T="ab**cac**"



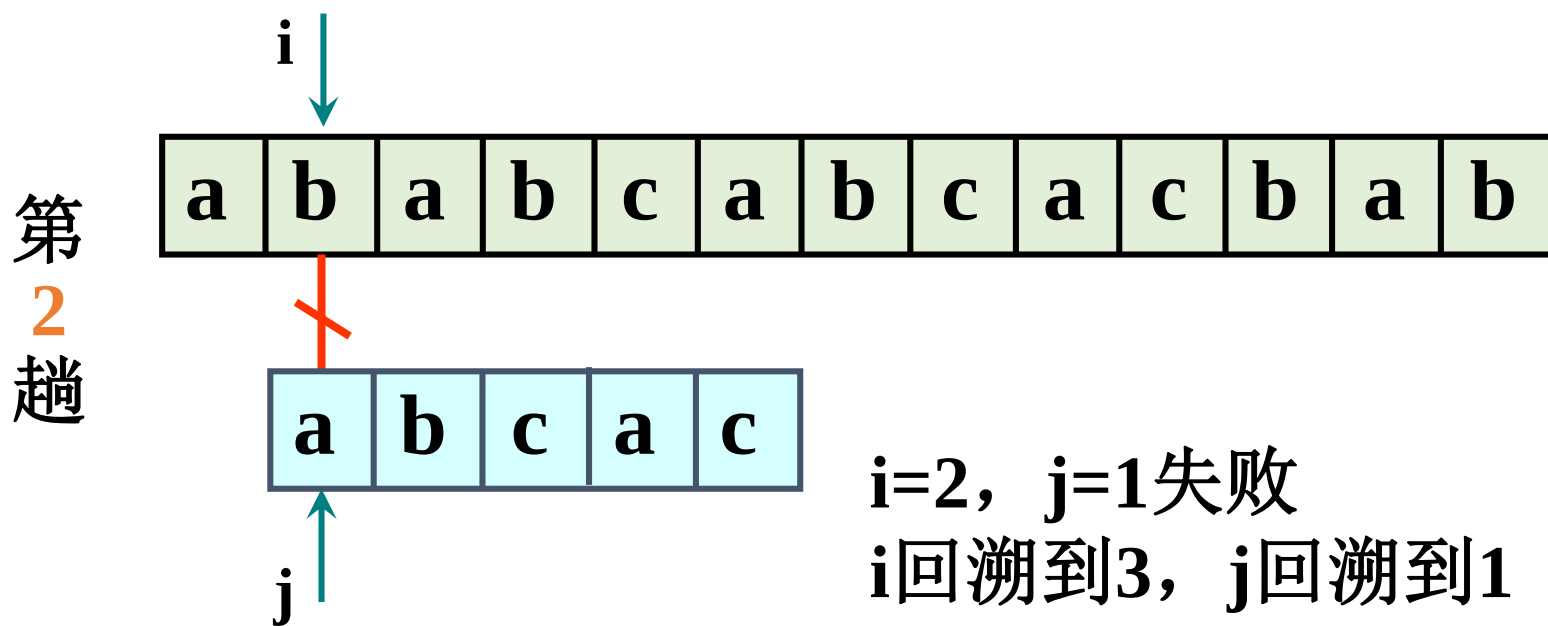
模式匹配——BF算法

例：主串S="ababcabcacbab"，模式T="abcac"



模式匹配——BF算法

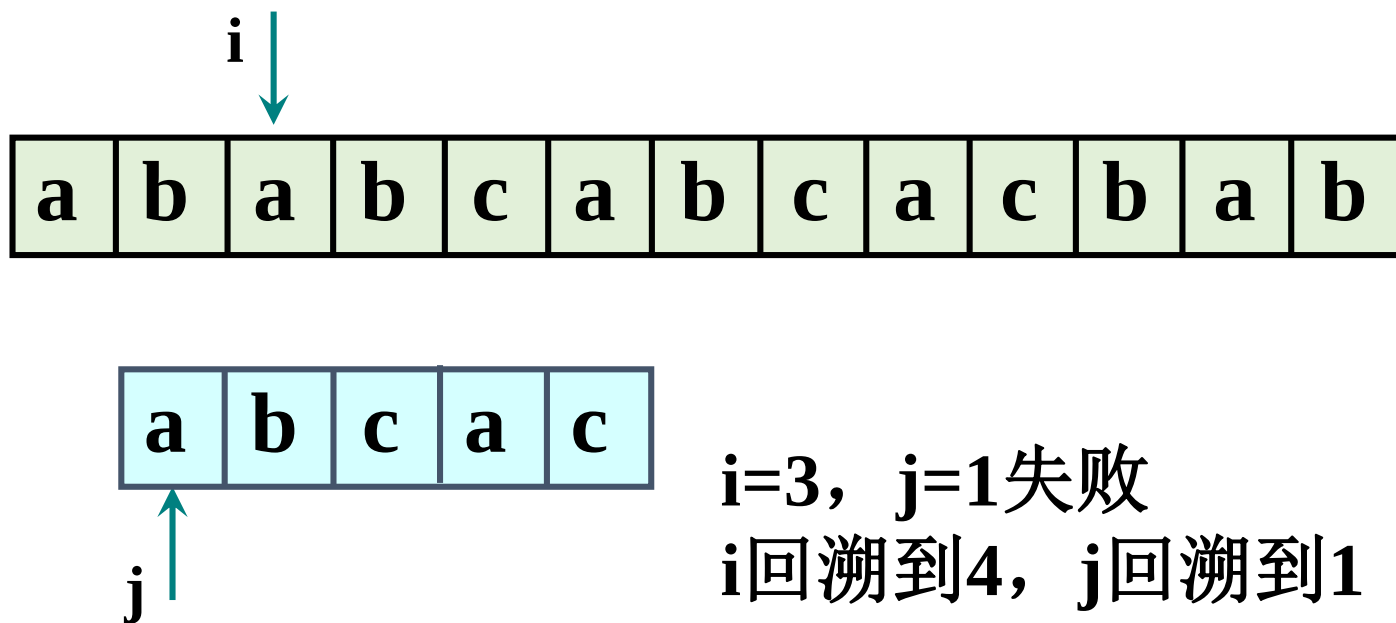
例：主串S="ababcabcacbab"，模式T="abcac"



模式匹配——BF算法

例：主串S="ababcabcacbab"，模式T="abcac"

第
2
趟



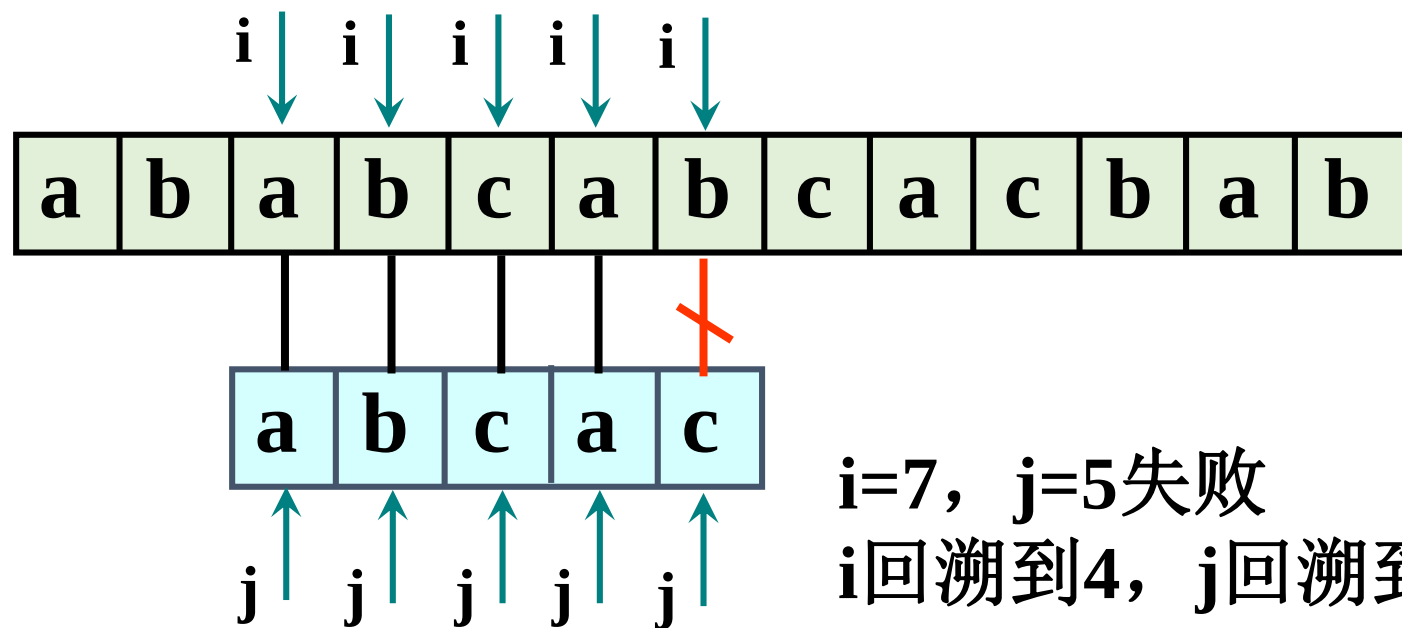
i=3, j=1失败

i回溯到4, j回溯到1

模式匹配——BF算法

例：主串S="ababcabcacbab"，模式T="abcac"

第
3
趟

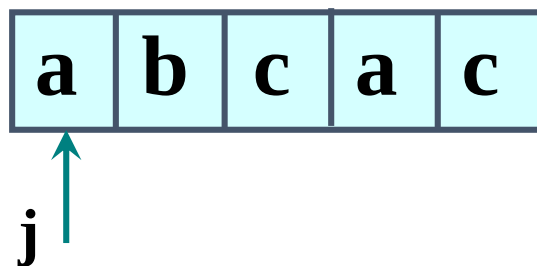
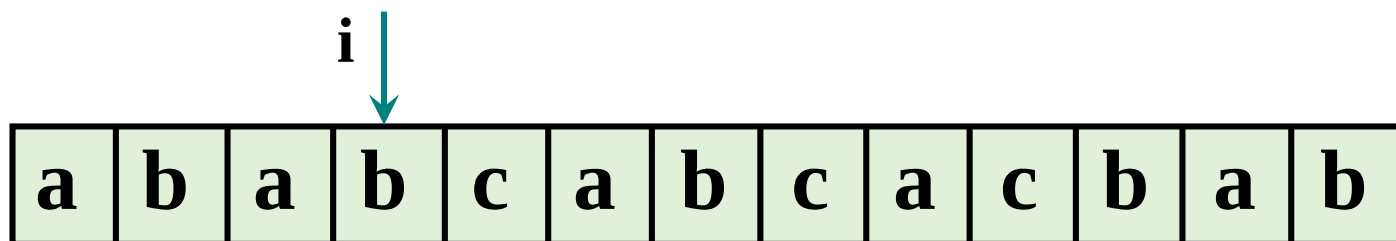


i=7, j=5失败
i回溯到4, j回溯到1

模式匹配——BF算法

例：主串S="ababcabcacbab"，模式T="abcac"

第
3
趟

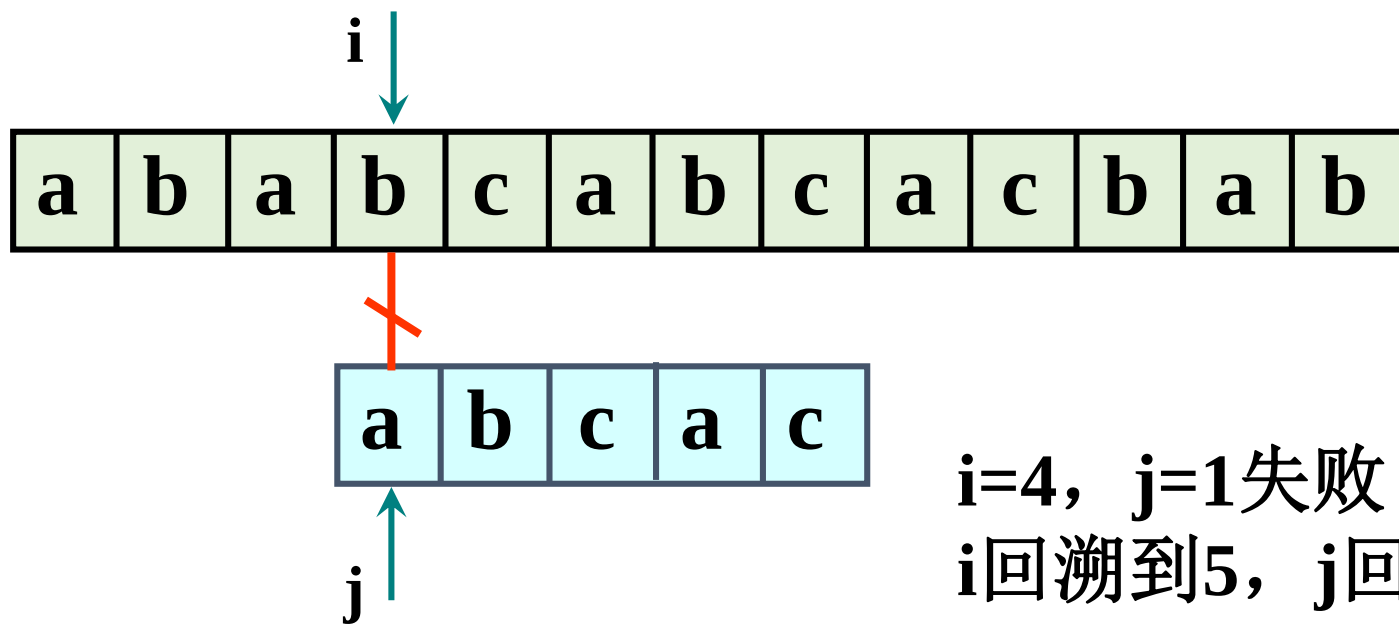


i=6, j=4失败
i回溯到3, j回溯到0

模式匹配——BF算法

例：主串S="ababcabcacbab"，模式T="abcac"

第
4
趟

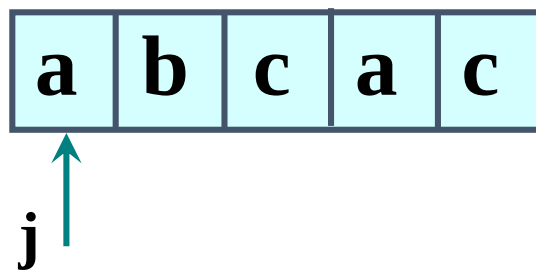
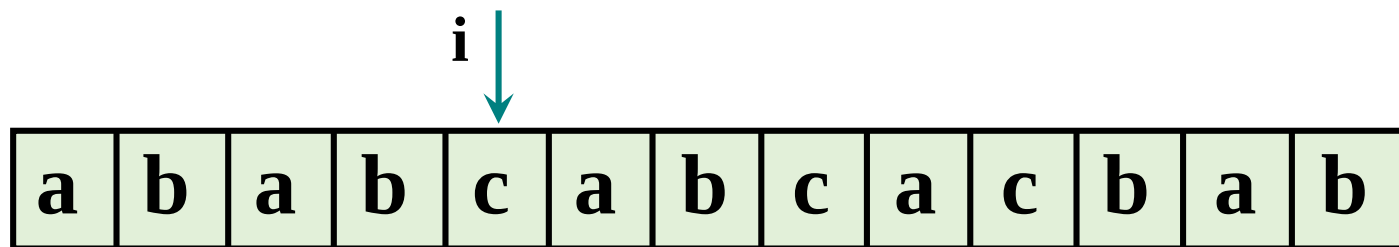


$i=4, j=1$ 失败
 i 回溯到5, j 回溯到1

模式匹配——BF算法

例：主串S="ababcabcacbab"，模式T="abcac"

第
4
趟

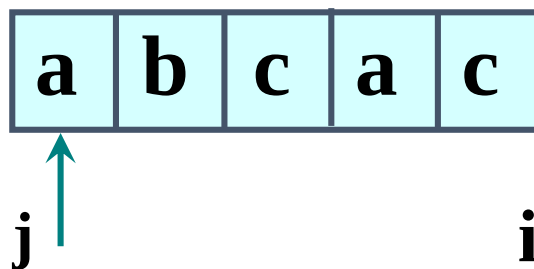
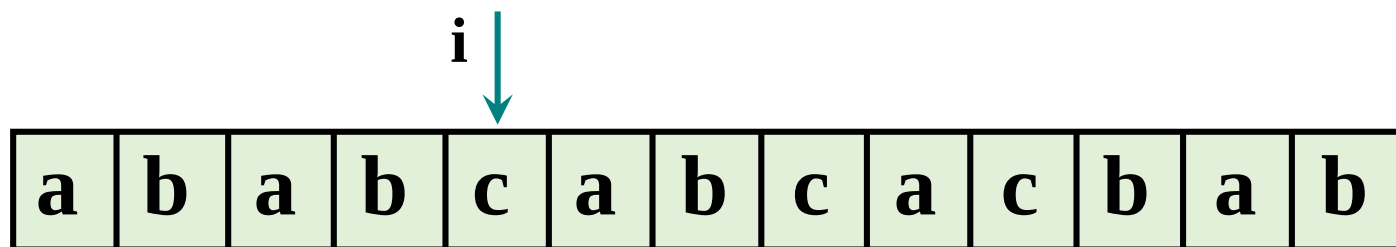


i=5, j=1失败
i回溯到6, j回溯到1

模式匹配——BF算法

例：主串S="ababcabcacbab"，模式T="abcac"

第
5
趟



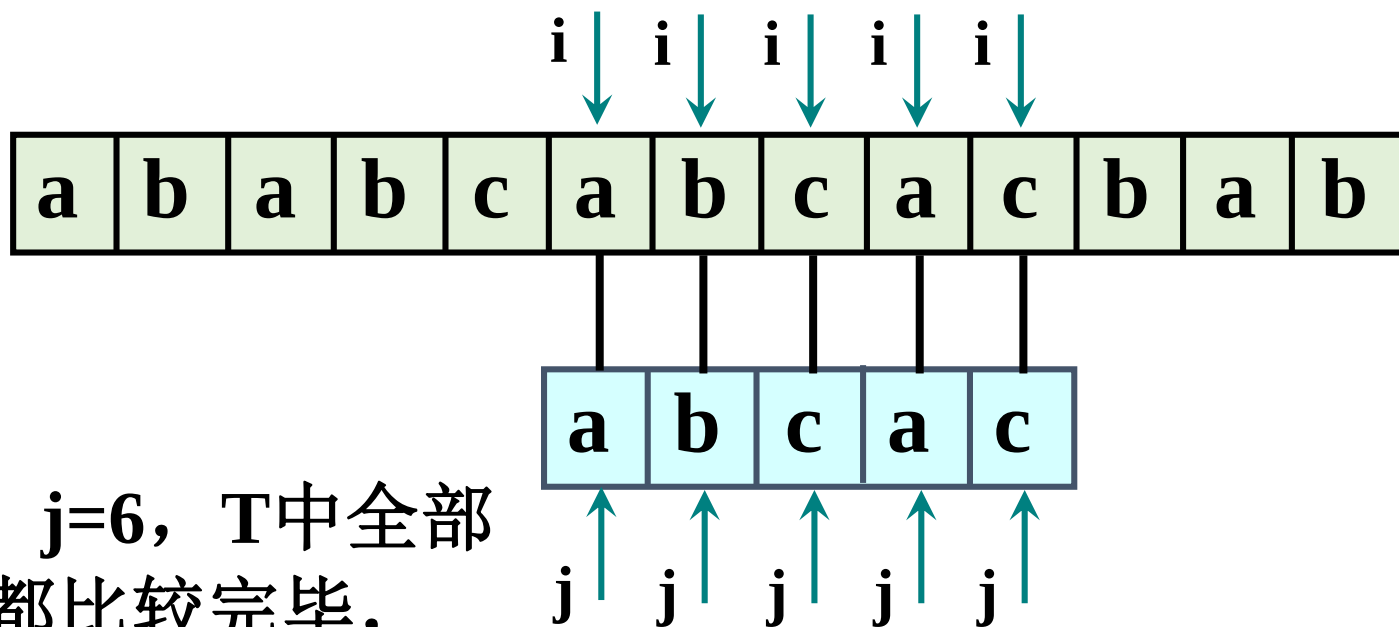
i=5, j=1失败
i回溯到6, j回溯到1

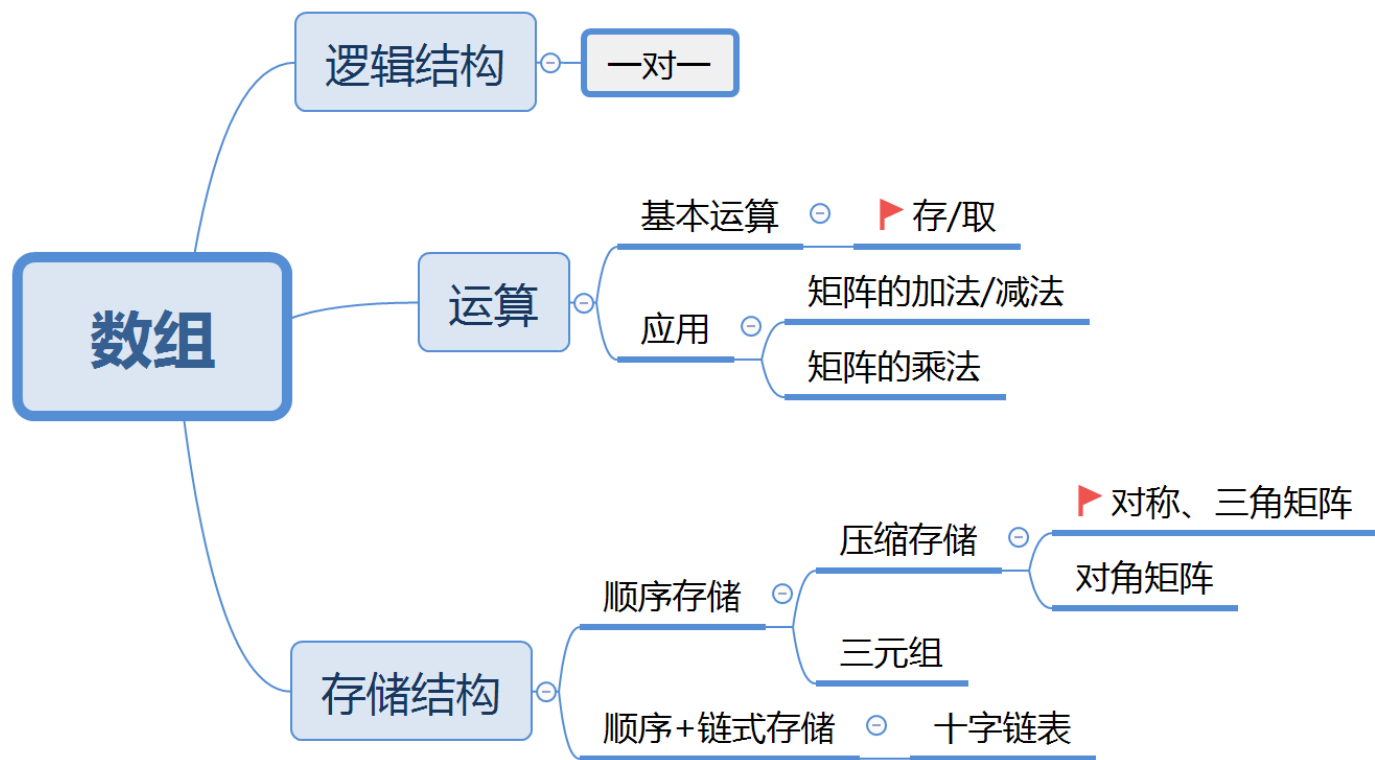
模式匹配——BF算法

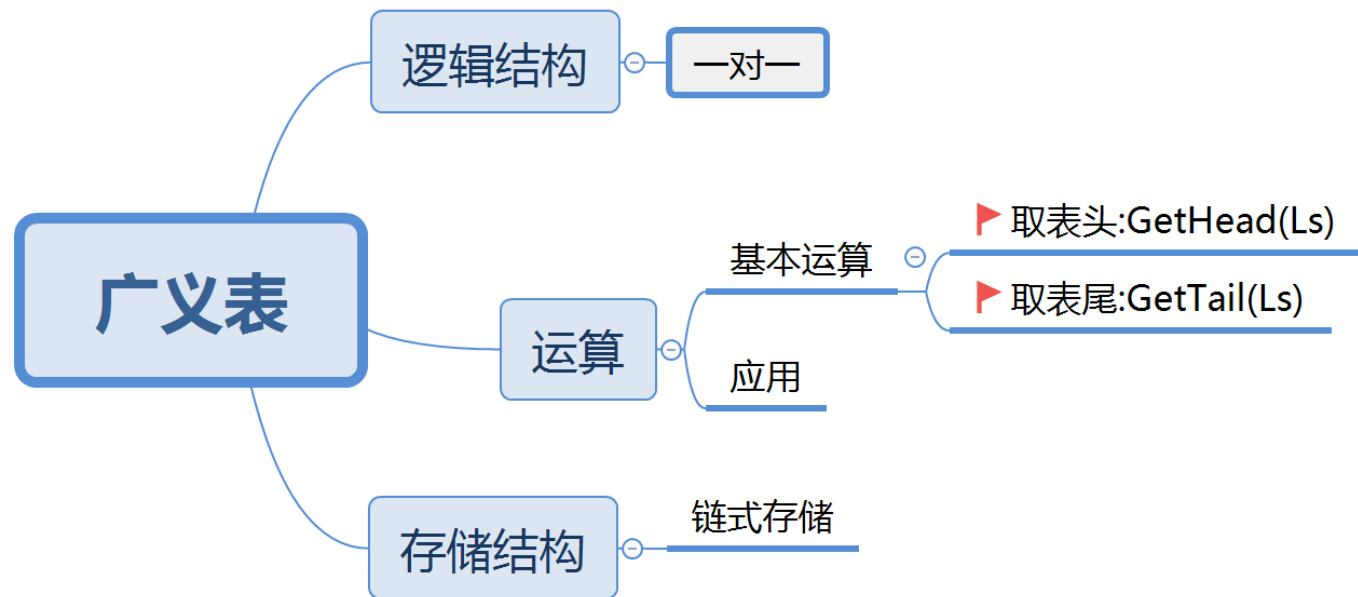
例：主串S="ababcabcacbab"，模式T="abcac"

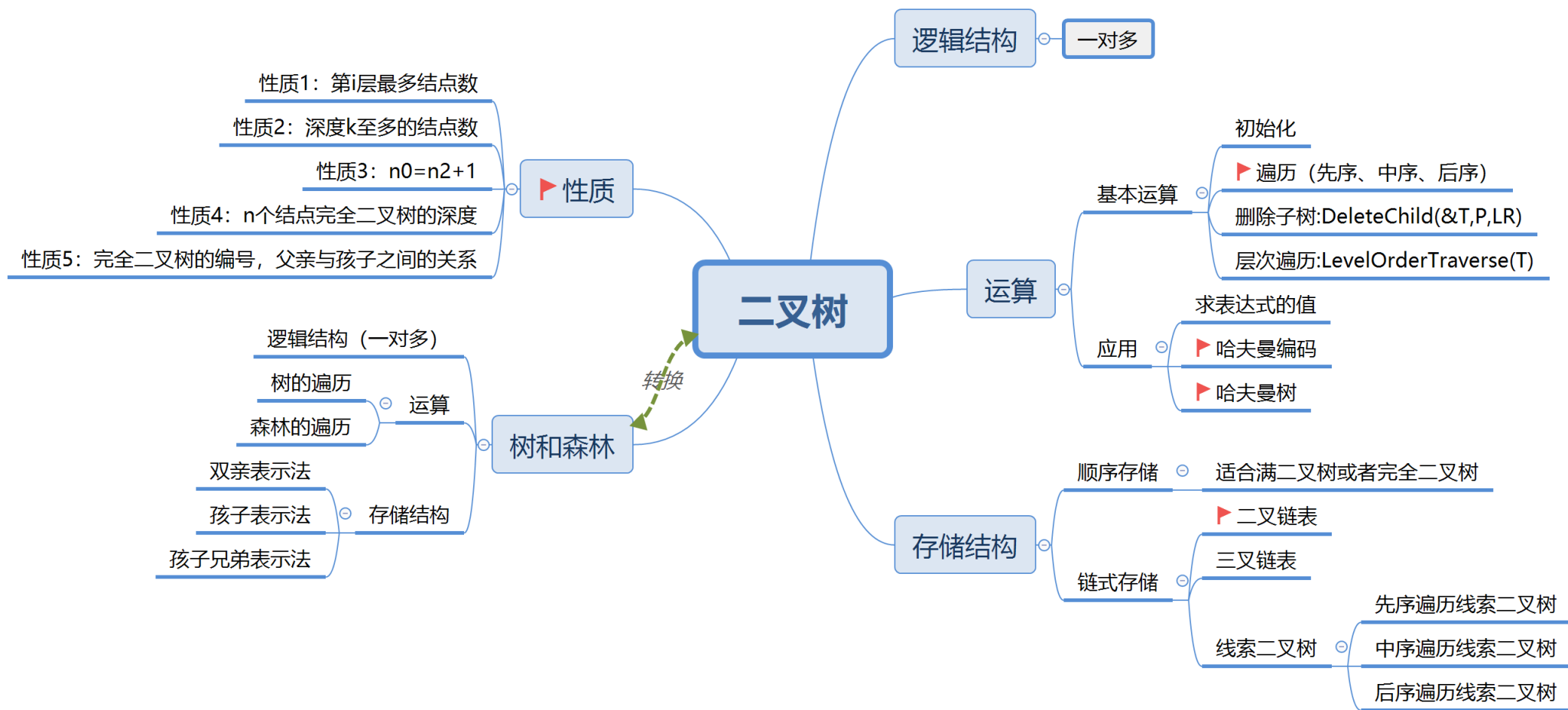
第
6
趟

$i=11$ ， $j=6$ ，T中全部
字符都比较完毕，
匹配成功





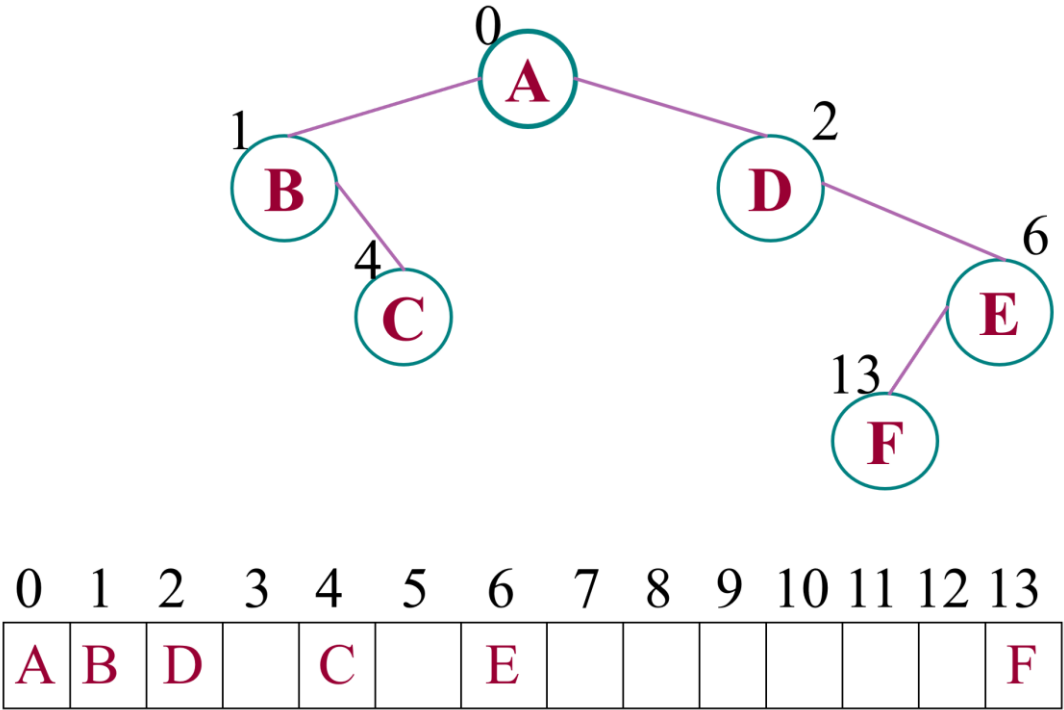


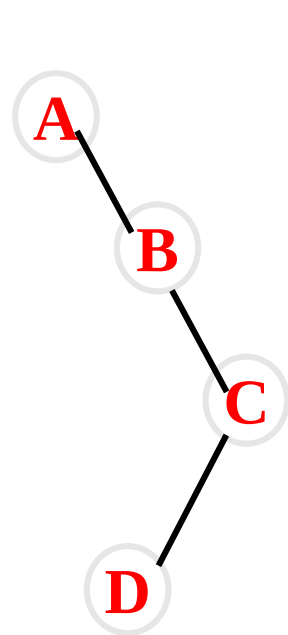


二叉树（顺序存储）

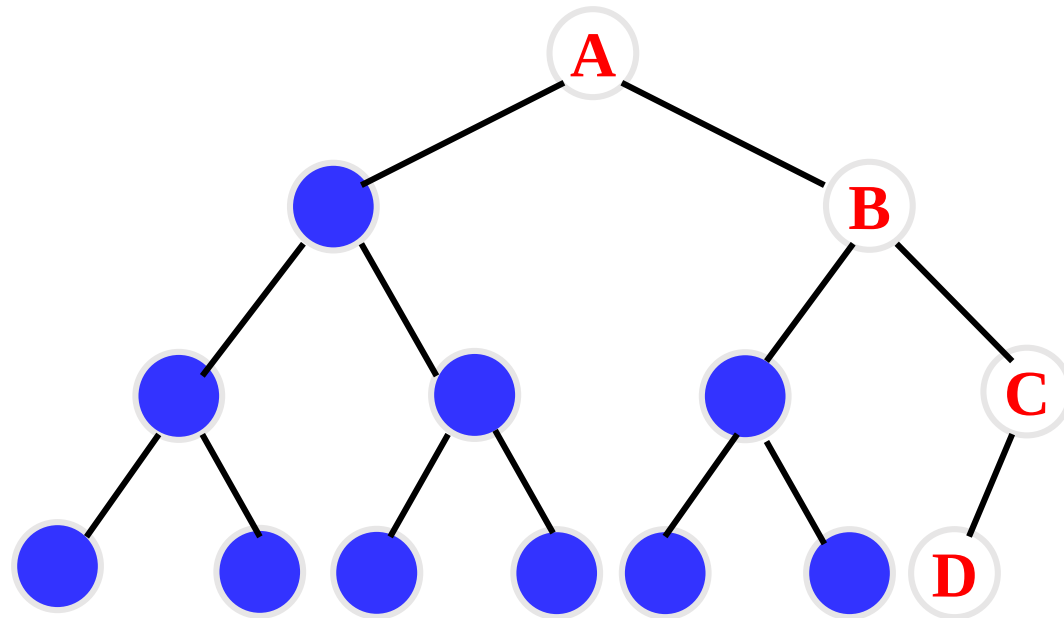
用一组连续的存储单元依次自上而下、自左而右存储完全二叉树上的结点元素。

```
#define MAX_TREE_SIZE 100 // 二叉树的最大结点数
typedef TElemType SqBiTree[MAX_TREE_SIZE]; // 0号单元存储根结点
SqBiTree bt;
```





一棵4个结点的二叉树



分配14个存储单元

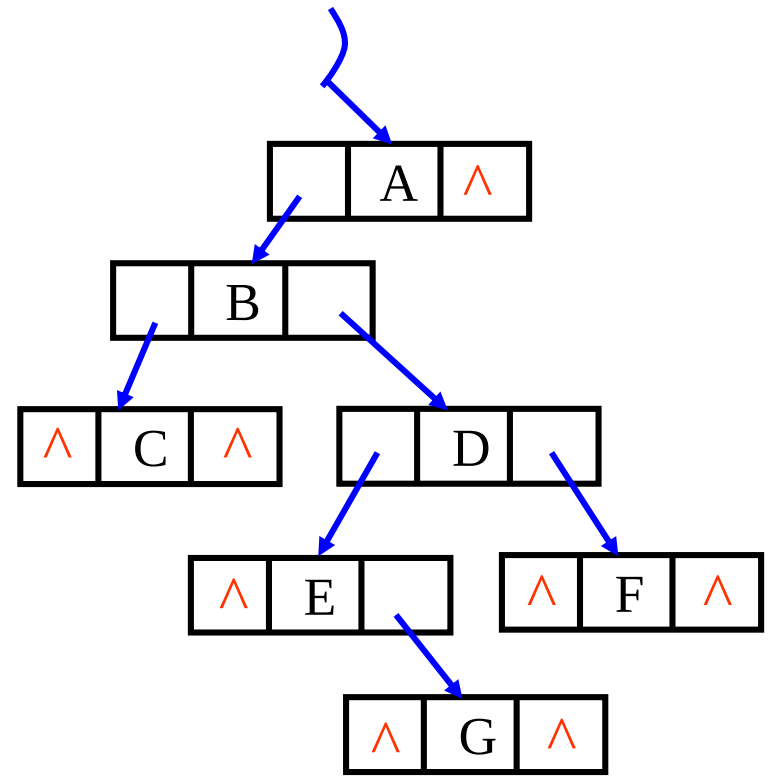
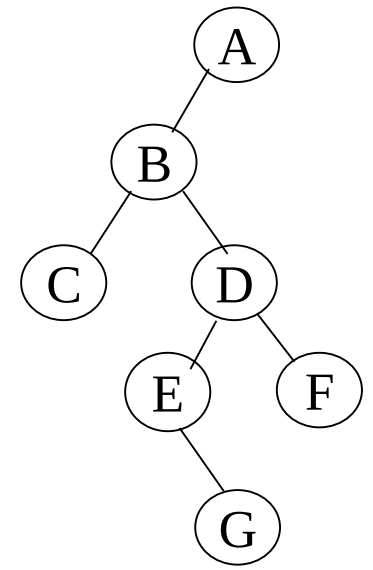
结点编号: 1 2 3 4 5 6 7 8 9 10 11 12 13 14

A	^	B	^	^	^	C	^	^	^	^	^	^	D
---	---	---	---	---	---	---	---	---	---	---	---	---	---

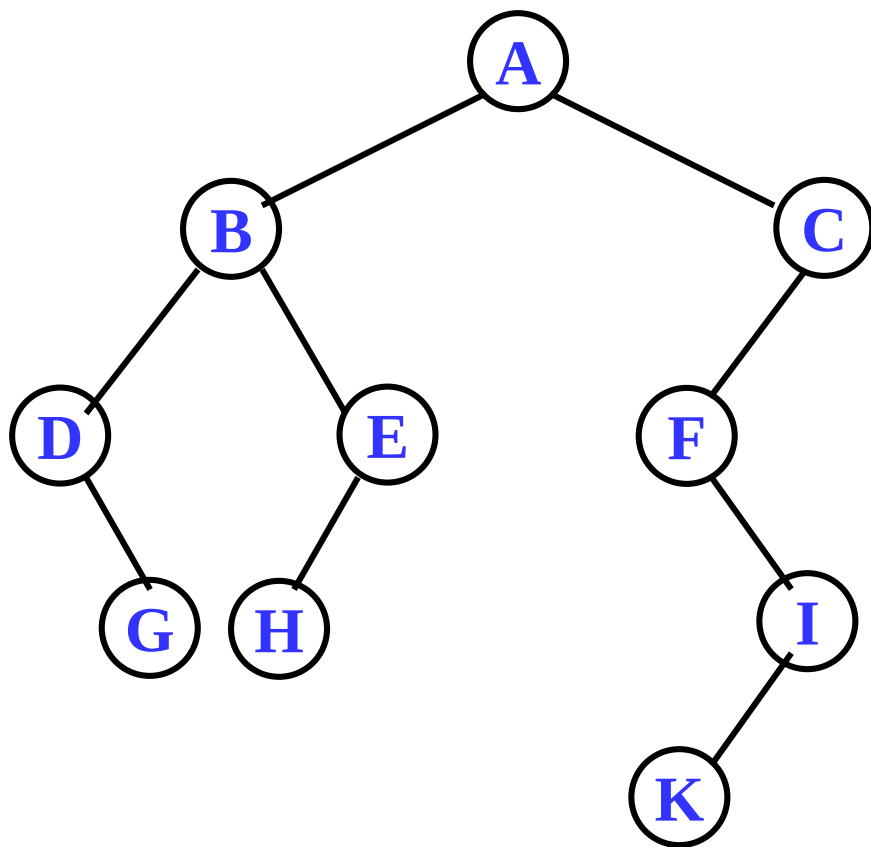
数组下标: 0 1 2 3 4 5 6 7 8 9 10 11 12 13

二叉链表

```
typedef struct BiNode{  
    TElemType  data;  
    struct BiNode  *lchild,*rchild;  
}BiNode,*BiTree;
```



如图所示，求二叉树的先序遍历、中序遍历、后序遍历和层次遍历。



先序遍历的序列：（**中**左右）

A B D G E H C F I K

中序遍历的序列：（左**中**右）

D G B H E A F K I C

后序遍历的序列：（左右**中**）

G D H E B K I F C A

层次遍历的序列：**上到下，左到右**

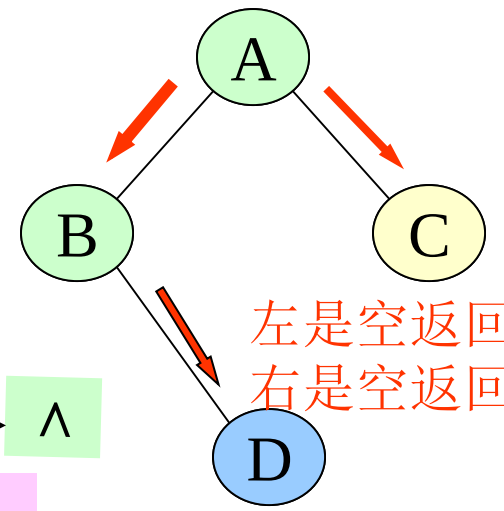
A B C D E F G H I K

先序遍历算法

```
Status PreOrderTraverse(BiTree T){  
    if(T==NULL) return OK;  
    else{  
        cout<<T->data;  
        PreOrderTraverse(T->lchild);  
        PreOrderTraverse(T->rchild); }  
}
```

左是空返回

左是空返回
右是空返回



主程序

Pre(T)

T → A

cout<<A;
pre(T→L);
pre(T→R);

T → B

cout<<B;
pre(T→L);
pre(T→R);

T → C

cout<<C;
pre(T→L);
pre(T→R);

T → Λ

返回

T → D

cout<<D;
pre(T→L);
pre(T→R);

T → Λ

返回

T → Λ

返回

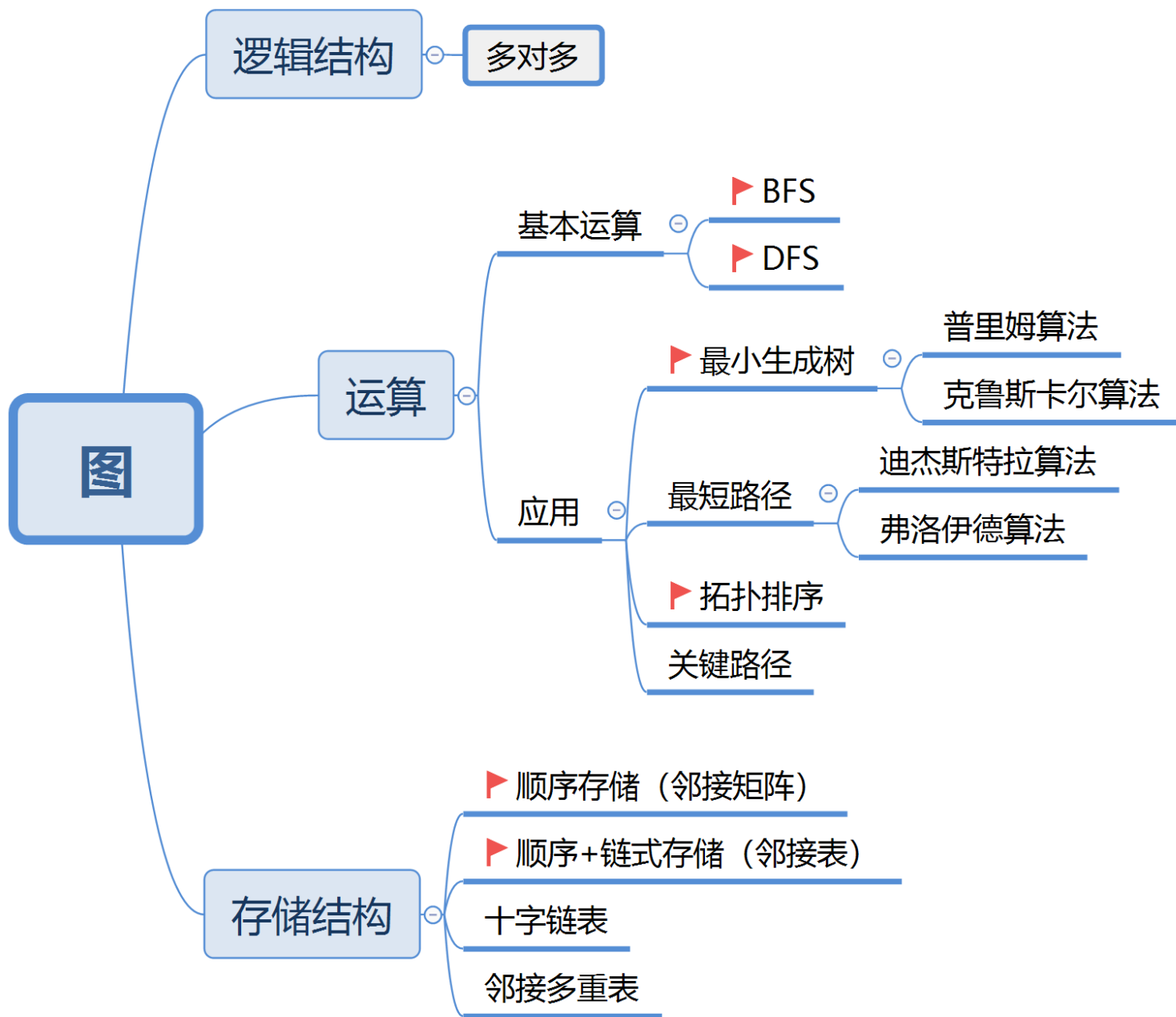
T → Λ

返回

T → Λ

返回

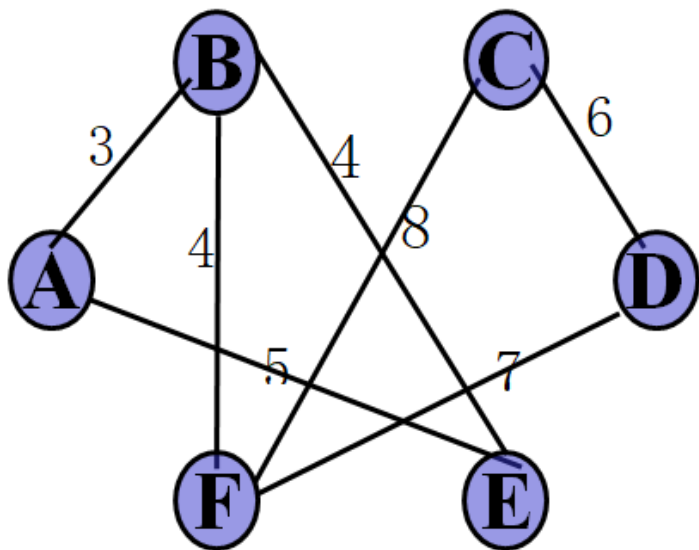
先序序列: **ABDC**



一、图的邻接矩阵存储表示

定义:矩阵的元素为

$$A_{ij} = \begin{cases} \infty & (i,j) \notin VR \\ \text{权值} & (i,j) \in VR \end{cases}$$



	A	B	C	D	E	F
A	∞	3	∞	∞	5	∞
B	3	∞	∞	∞	4	4
C	∞	∞	∞	6	∞	8
D	∞	∞	6	∞	∞	7
E	5	4	∞	∞	∞	∞
F	∞	4	8	7	∞	∞

邻接矩阵的存储表示

//用两个数组分别存储顶点表和邻接矩阵

```
#define MaxInt 32767
```

//表示极大值，即 ∞

```
#define MVNum 100
```

//最大顶点数

```
typedef char VerTexType;
```

//假设顶点的数据类型为字符型

```
typedef int ArcType;
```

//假设边的权值类型为整型

```
typedef struct
```

```
{
```

```
    VerTexType vexs[MVNum];
```

//顶点表

```
    ArcType arcs[MVNum][MVNum];
```

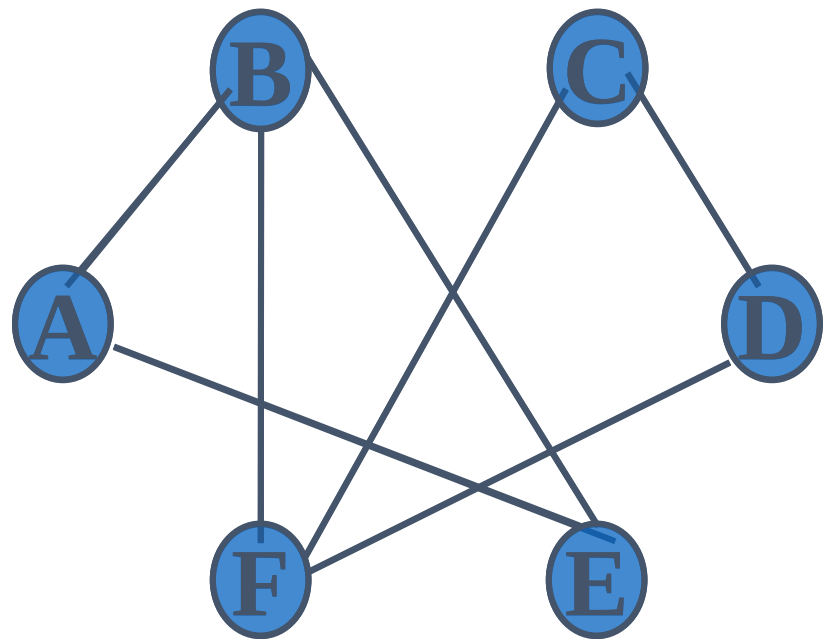
//邻接矩阵

```
    int vexnum,arcnum;
```

//图的当前点数和边数

```
}AMGraph;
```

二、图的邻接表存储表示



0	A	→	4	→	1	∧
1	B	→	5	→	4	→ 0 ∧
2	C	→	5	→	3	∧
3	D	→	5	→	2	∧
4	E	→	1	→	0	∧
5	F	→	3	→	2	→ 1 ∧

邻接表的存储表示

```
#define MVNum 100
typedef struct ArcNode{
    int adjvex;
    struct ArcNode * nextarc;
    OtherInfo info;
}ArcNode;
```

```
typedef struct VNode{
    VerTexType data;
    ArcNode * firstarc;
}VNode, AdjList[MVNum];
```

```
typedef struct{
    AdjList vertices;
    int vexnum, arcnum;
}ALGraph;
```

//最大顶点数

//边结点

//该边所指向的顶点的位置

//指向下一条边的指针

//和边相关的信息

//顶点信息

//指向第一条依附该顶点的边的指针

//AdjList表示邻接表类型

//邻接表

//图的当前顶点数和边数

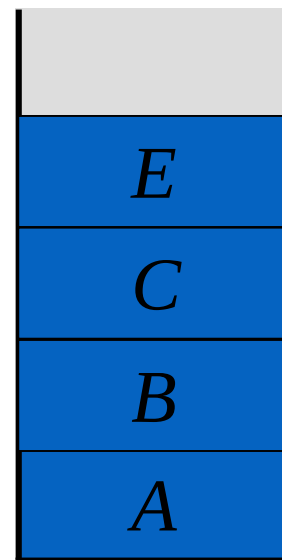
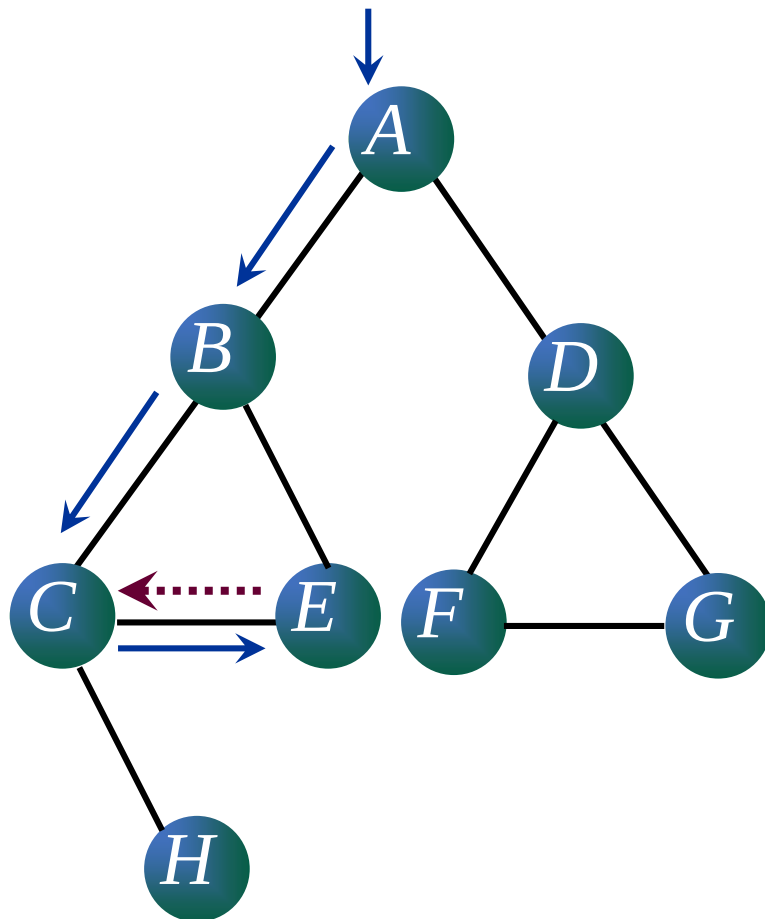
图的深度优先遍历 (DFS)

① 深度优先遍历序列?入栈序列?出栈序列?

深一层递归



递归返回



遍历序列: *A B C E*

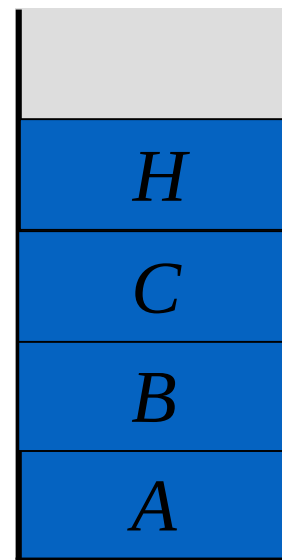
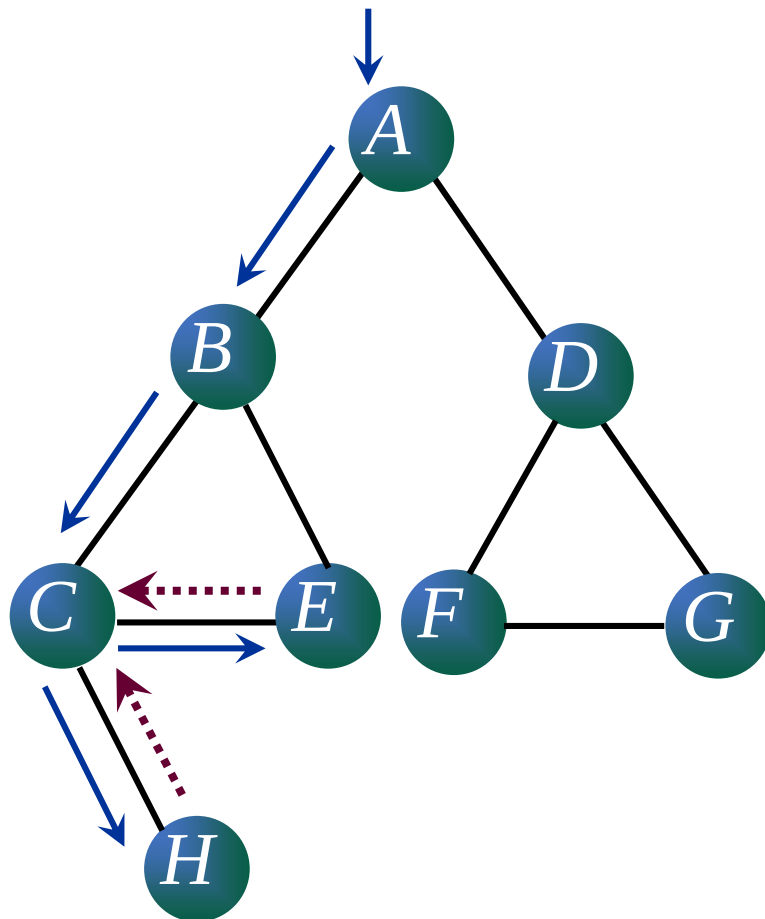
图的深度优先遍历 (DFS)

① 深度优先遍历序列?入栈序列?出栈序列?

深一层递归



递归返回



遍历序列: *A B C E H*

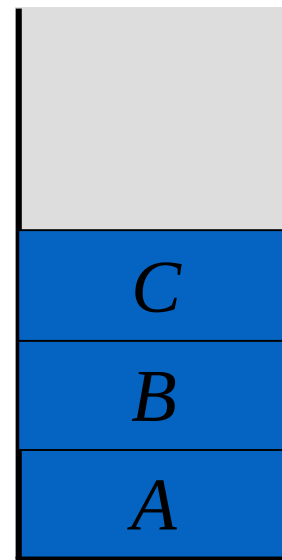
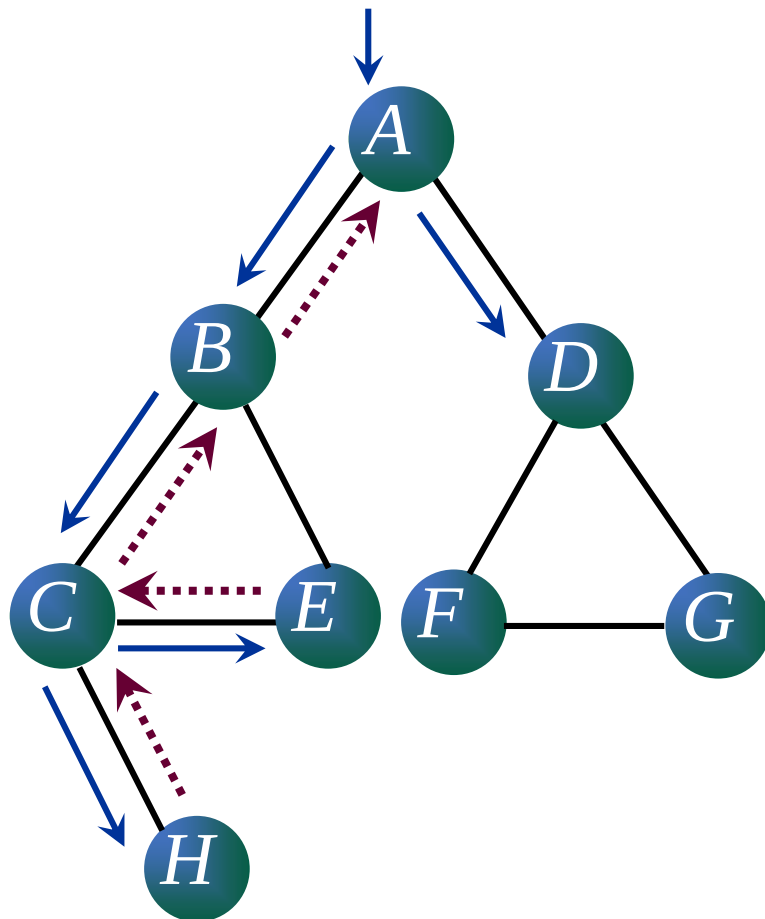
图的深度优先遍历 (DFS)

① 深度优先遍历序列?入栈序列?出栈序列?

深一层递归



递归返回



遍历序列: *A B C E H*

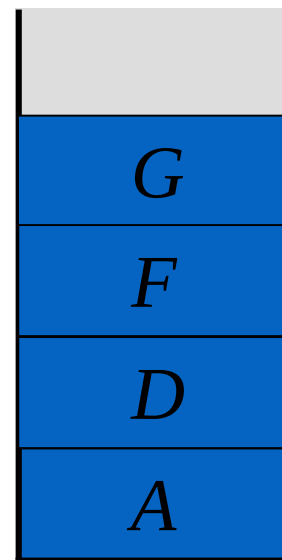
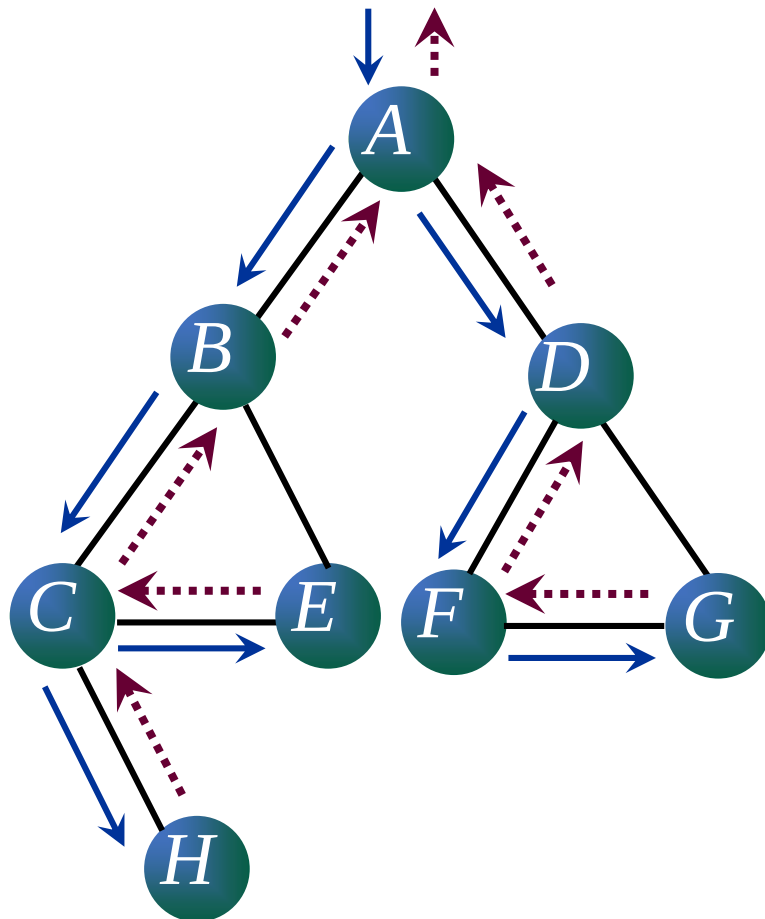
图的深度优先遍历 (DFS)

① 深度优先遍历序列?入栈序列?出栈序列?

深一层递归

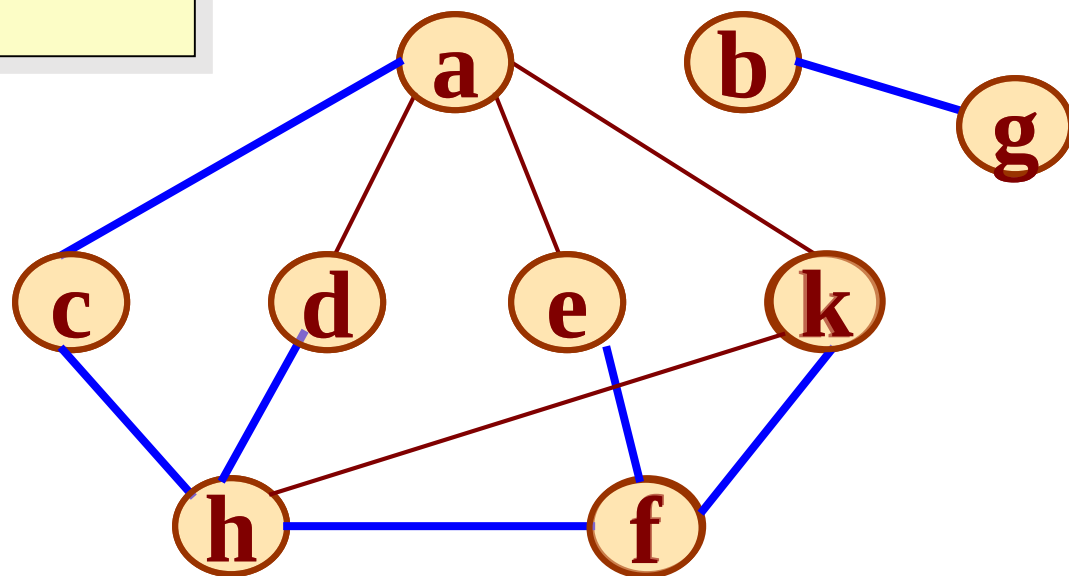


递归返回



遍历序列: *A B C E H D F G*

图的深度优先遍历 (DFS)



a	b	c	d	e	f	g	h	k
0	1	2	3	4	5	6	7	8

访问标志:

T	T	T	T	T	T	T	T	T
---	---	---	---	---	---	---	---	---

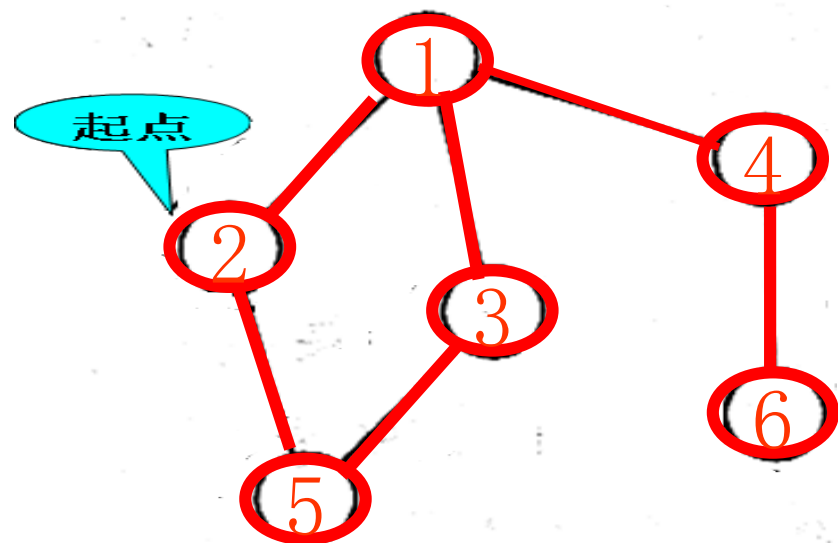
访问次序:

a	c	h	d	f	e	k	b	g
---	---	---	---	---	---	---	---	---

图的深度优先遍历 (DFS)

邻接矩阵 A

	1	2	3	4	5	6
1	0	1	1	1	0	0
2	1	0	0	0	1	0
3	1	0	0	0	1	0
4	1	0	0	0	0	1
5	0	1	1	0	0	0
6	0	0	0	1	0	0



辅助数组 *visited* [n]

	1	2	3	4	5	6
1	0	0	1	1	1	1
2	0	1	1	1	1	1
3	0	0	0	1	1	1
4	0	0	0	0	0	1
5	0	0	0	0	1	1
6	0	0	0	0	0	1

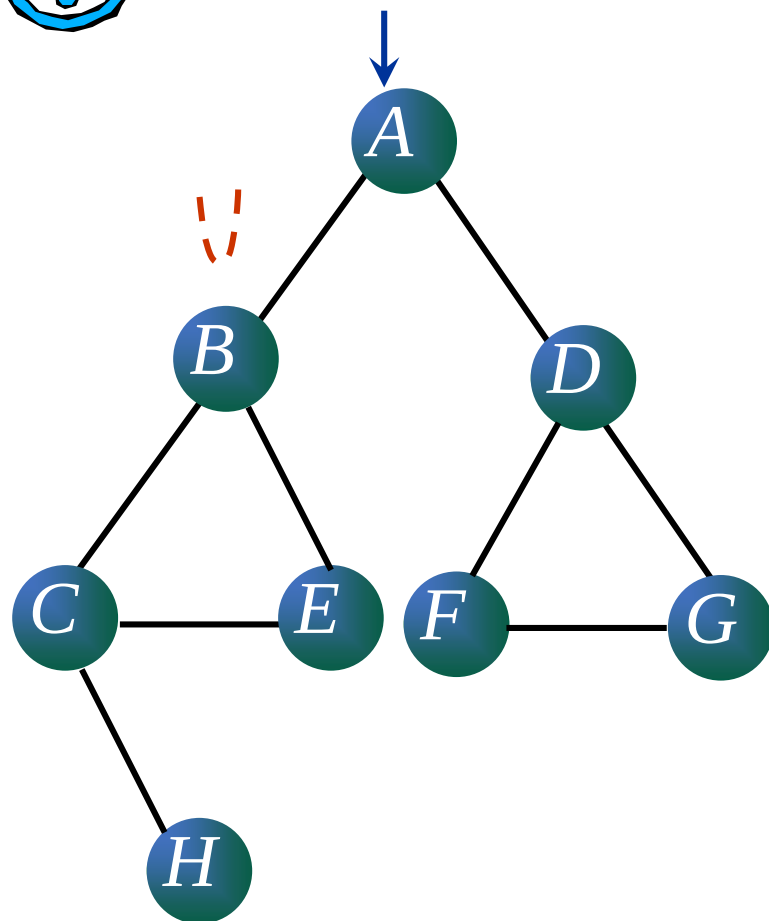
DFS 结果

2→1→3→5→4→6

图的广度优先遍历 (BFS)



广度优先遍历序列?入队序列?出队序列?

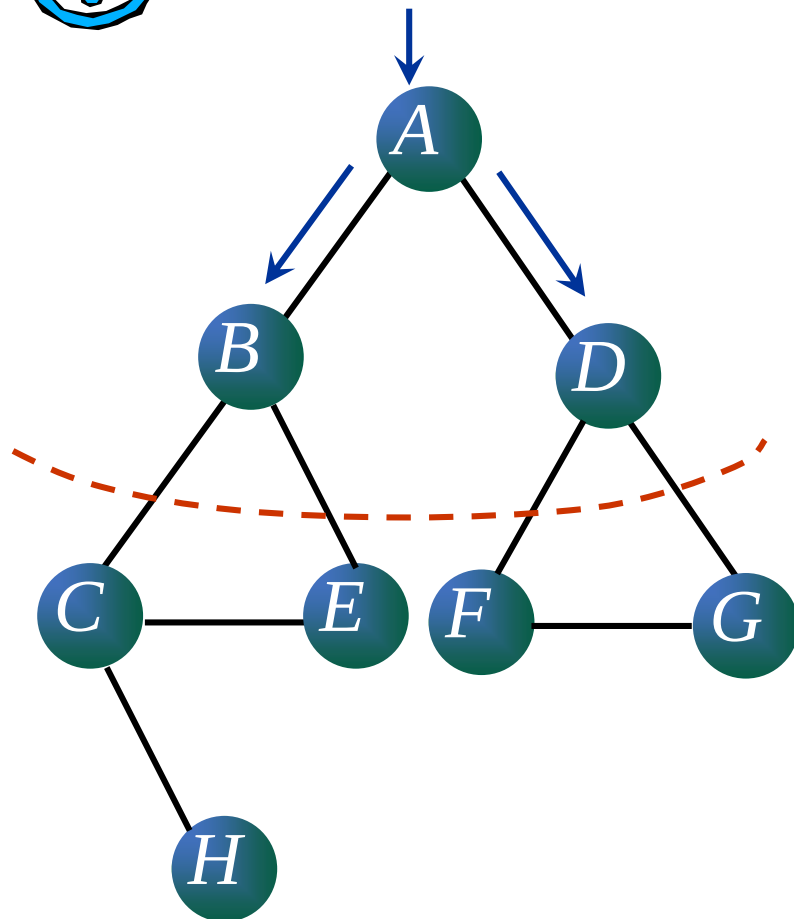


A

遍历序列: A

图的广度优先遍历 (BFS)

① 广度优先遍历序列?入队序列?出队序列?

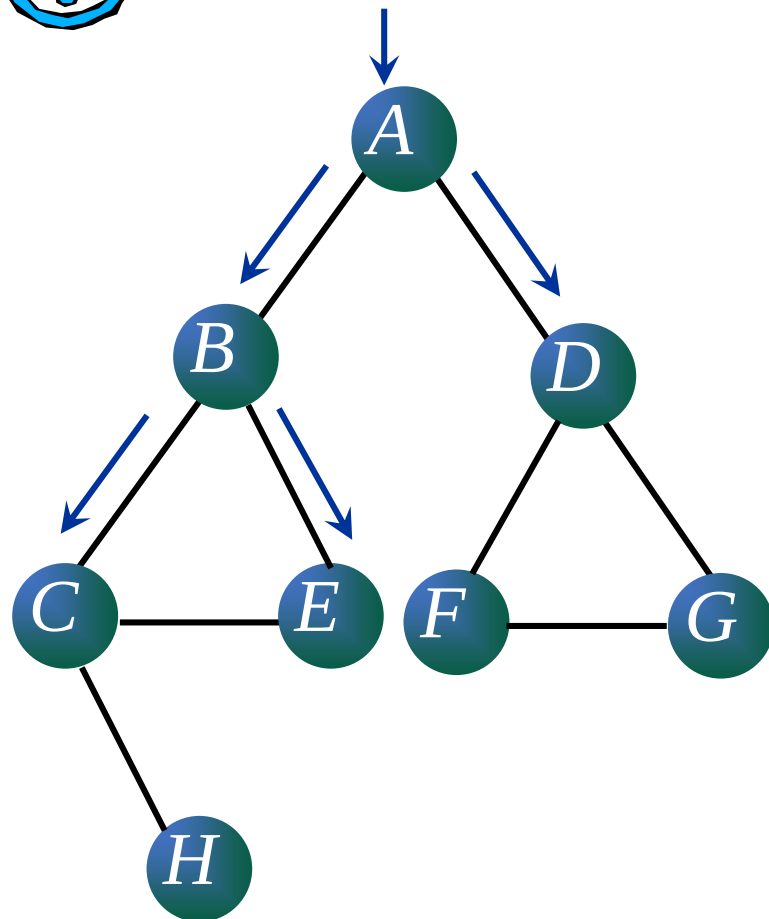


B D

遍历序列: *A B D*

图的广度优先遍历 (BFS)

① 广度优先遍历序列? 入队序列? 出队序列?



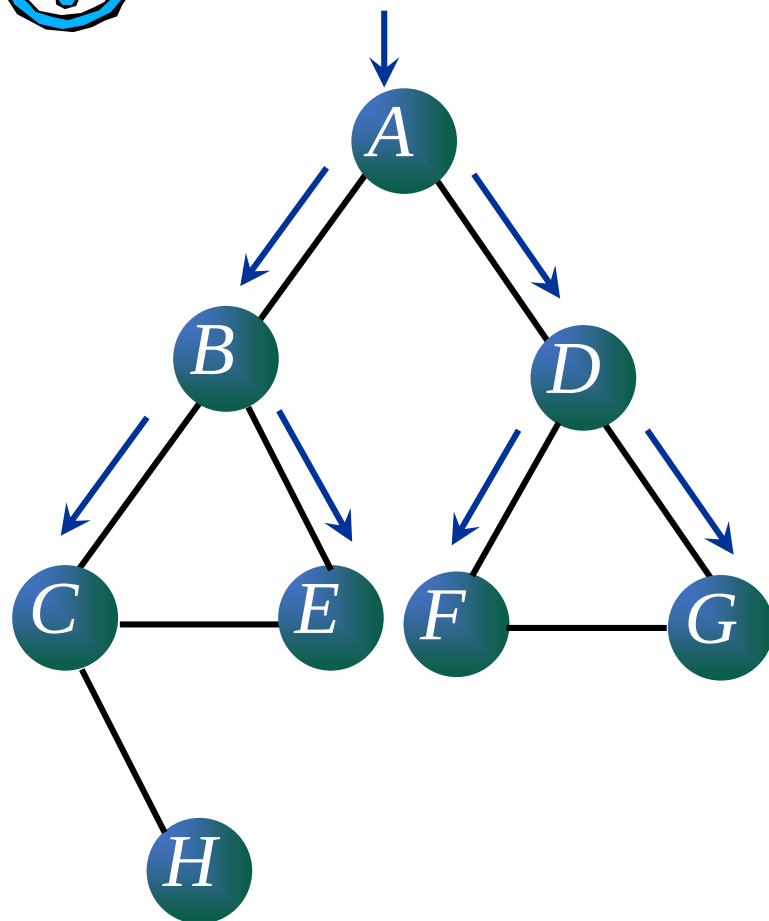
D C E

遍历序列: *A B D C E*

图的广度优先遍历 (BFS)



广度优先遍历序列?入队序列?出队序列?

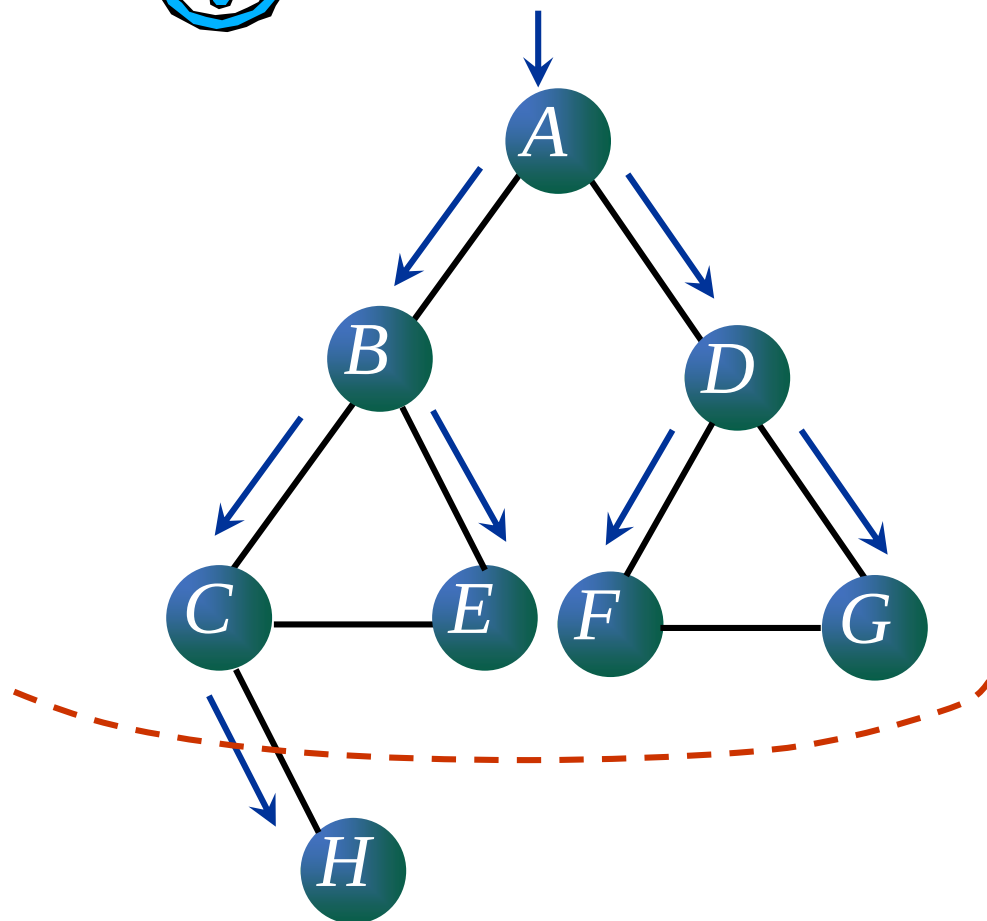


C E F G

遍历序列: *A B D C E F G*

图的广度优先遍历 (BFS)

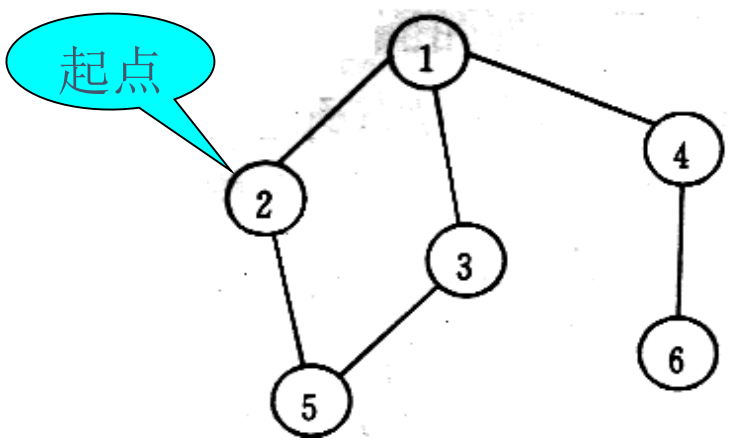
① 广度优先遍历序列? 入队序列? 出队序列?



E F G H

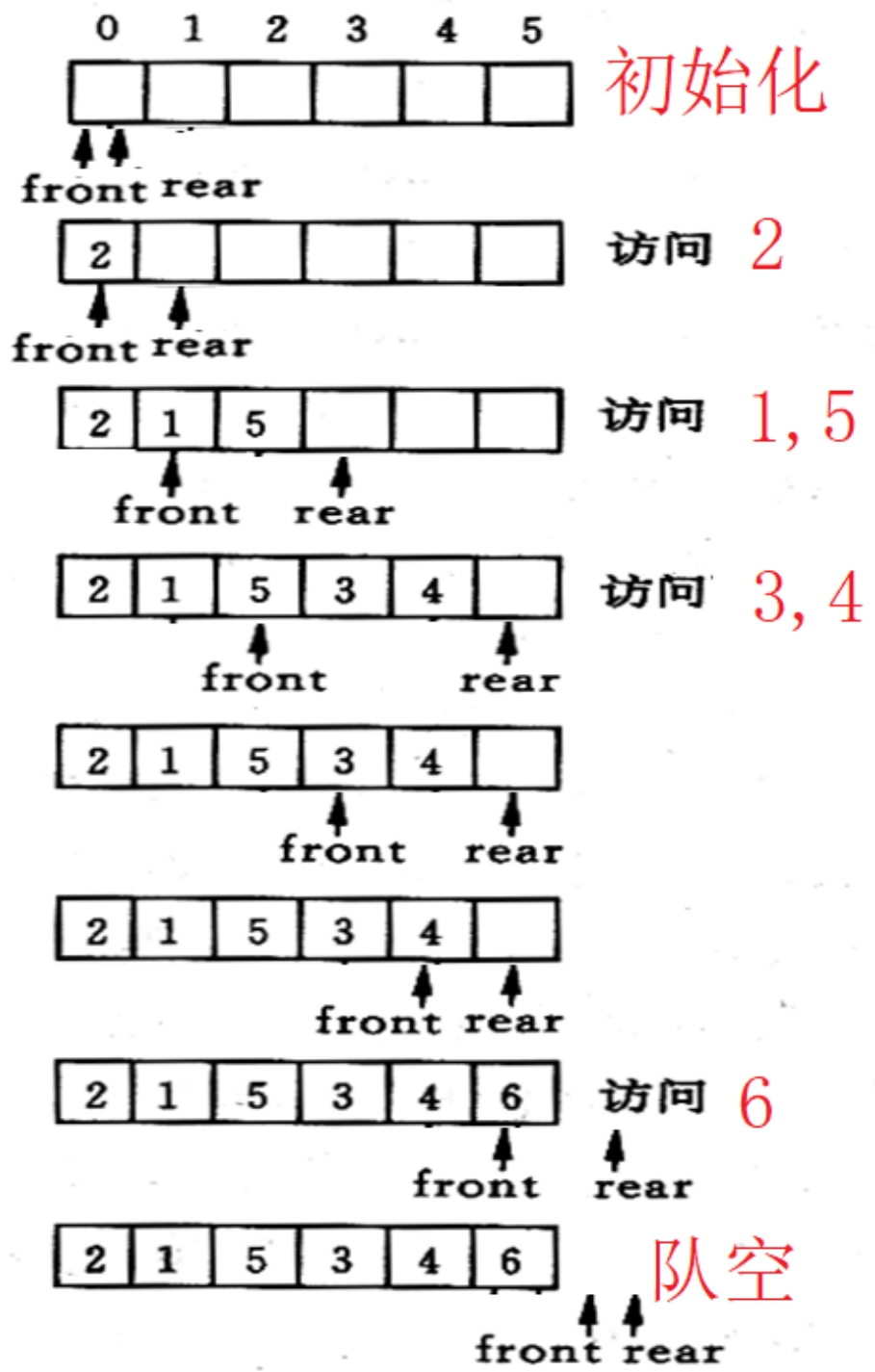
遍历序列: *A B D C E F G H*

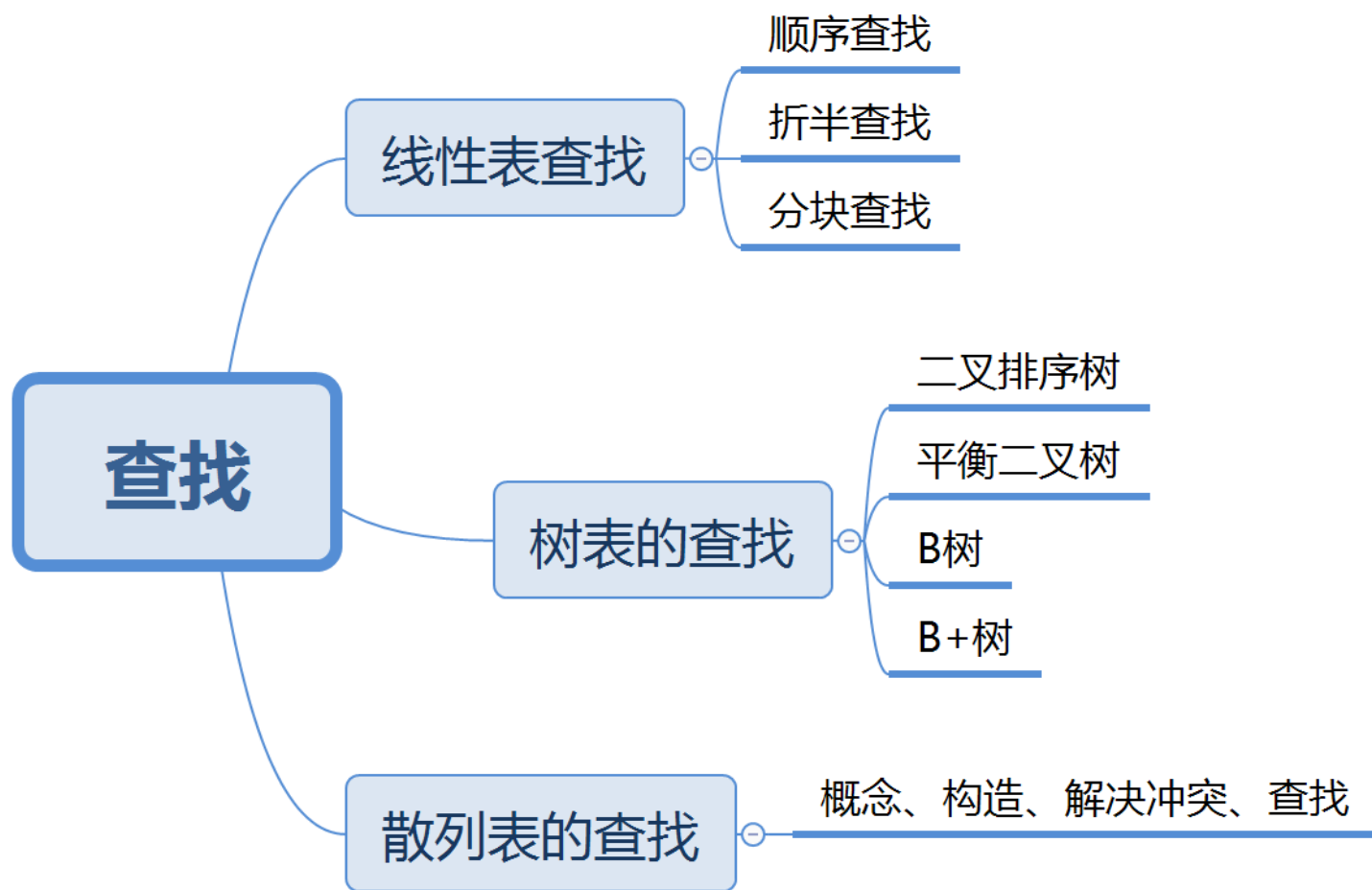
图的广度优先遍历 (BFS)

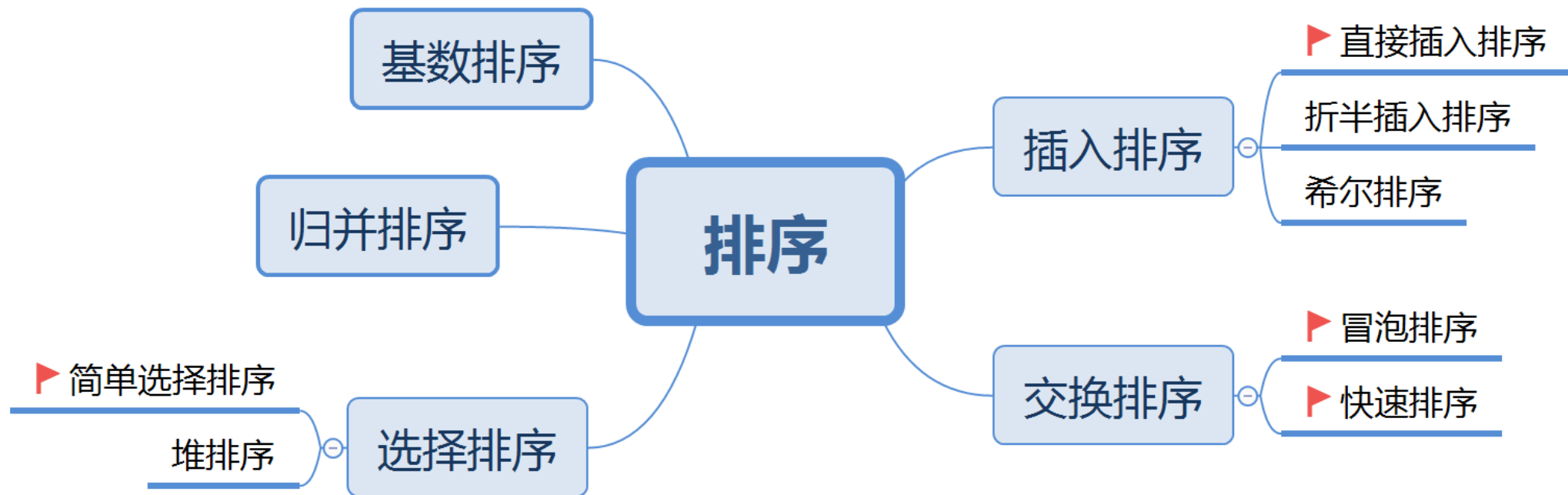


邻接矩阵 A

	1	2	3	4	5	6
1	0	1	1	1	0	0
2	1	0	0	0	1	0
3	1	0	0	0	1	0
4	1	0	0	0	0	1
5	0	1	1	0	0	0
6	0	0	0	1	0	0



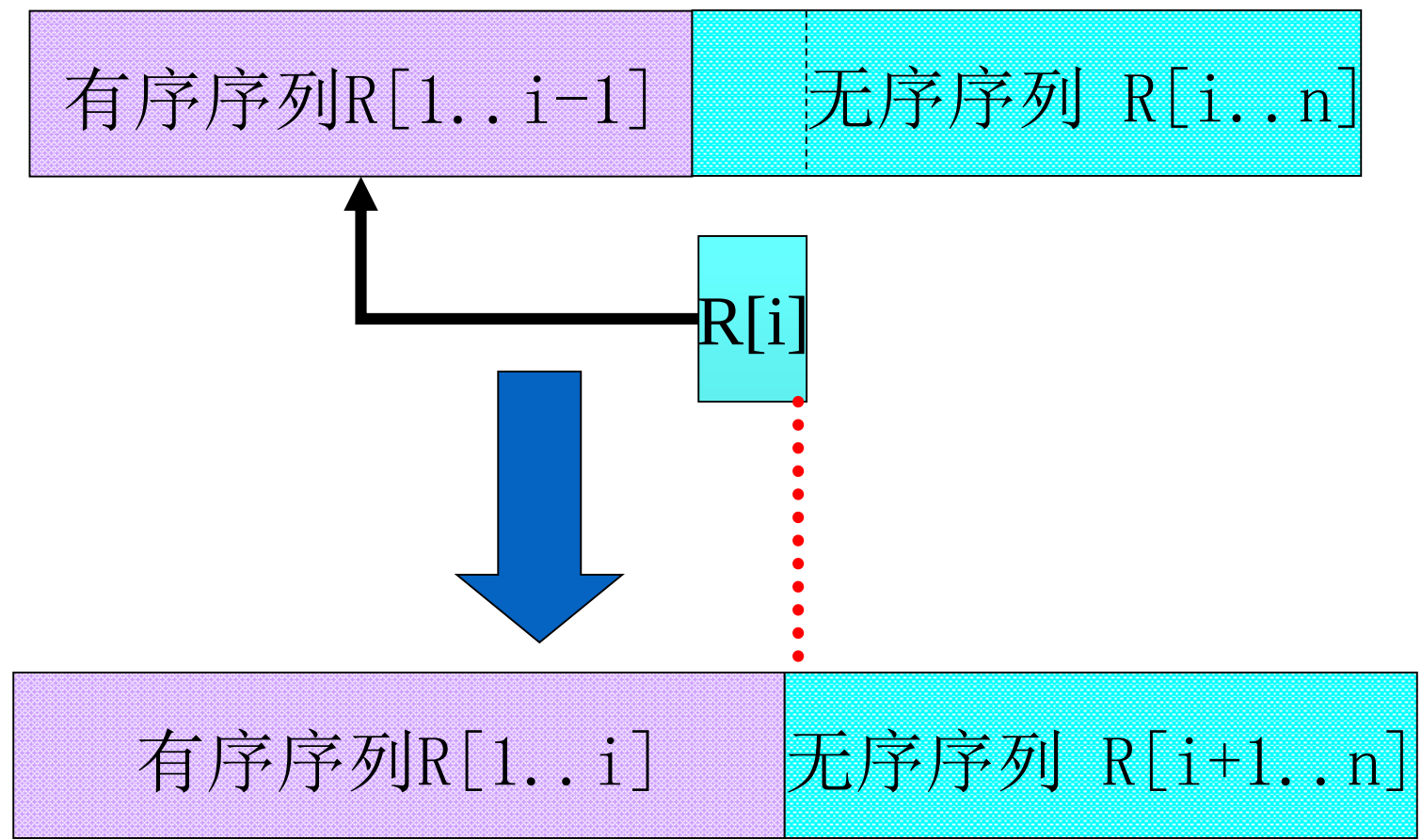




直接插入排序

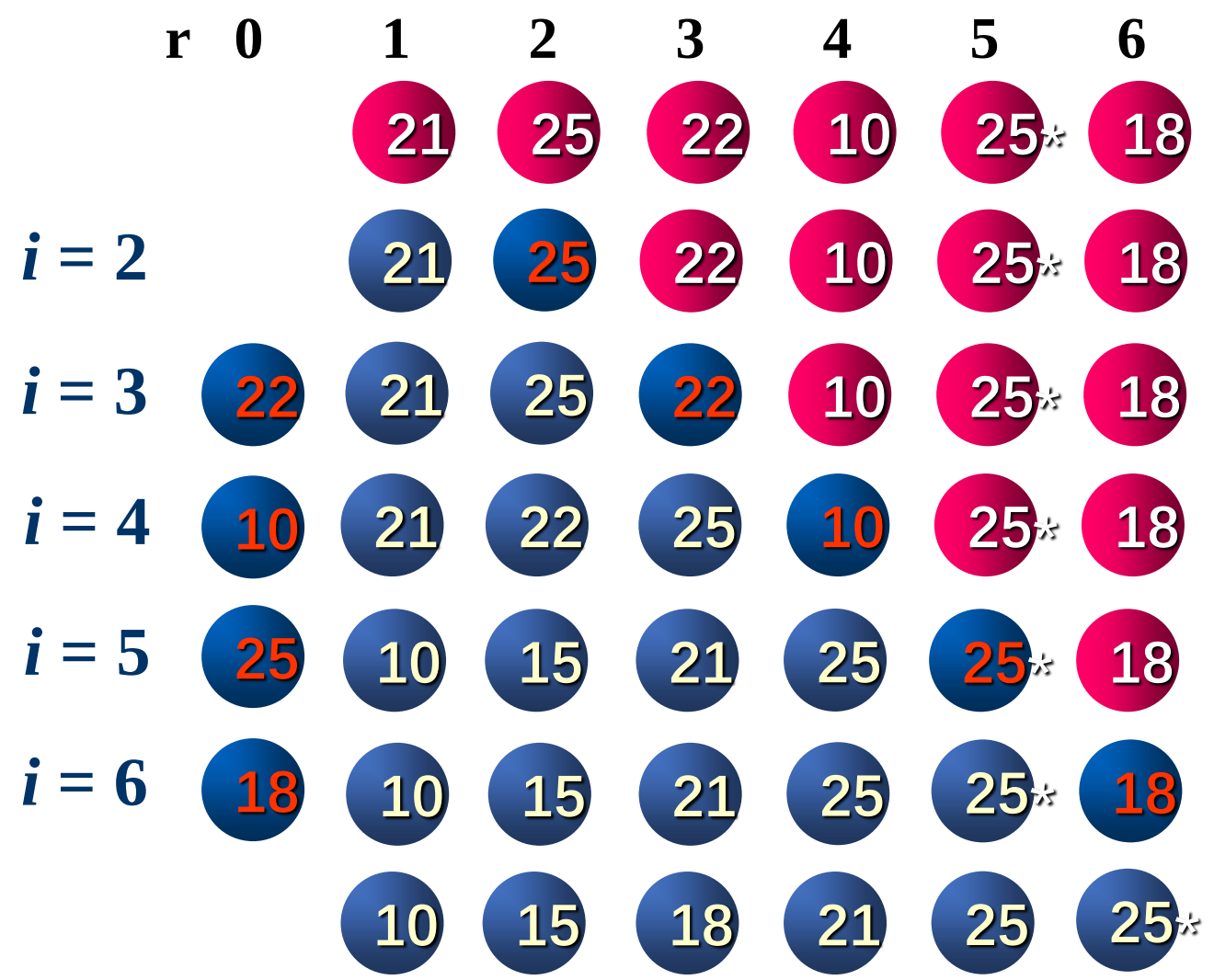
主要操作：插入

基本思想是：每次将一个待排序的第一个数插入到一个已经排好序的有序序列中，直到全部记录排好序为止。



直接插入排序过程示例

直接插入排序算法



```
void insertSort (int r[ ], int n)
{
    for (i=2; i<=n; i++)
    {
        r[0]=r[i]; j=i-1;
        while (r[0]<r[j])
        {
            r[j+1]=r[j];
            j--;
        }
        r[j+1]=r[0];
    }
}
```

① $r[0]$ 的作用? 暂存单元、监视哨

直接插入排序算法性能分析

最好情况下（正序）：



{ 比较次数: $n-1$
移动次数: $2(n-1)$

时间复杂度为 $O(n)$ 。

最坏情况下（逆序或反序）：



{ 比较次数: $\sum_{i=2}^n i = \frac{(n+2)(n-1)}{2}$
移动次数: $\sum_{i=2}^n (i+1) = \frac{(n+4)(n-1)}{2}$

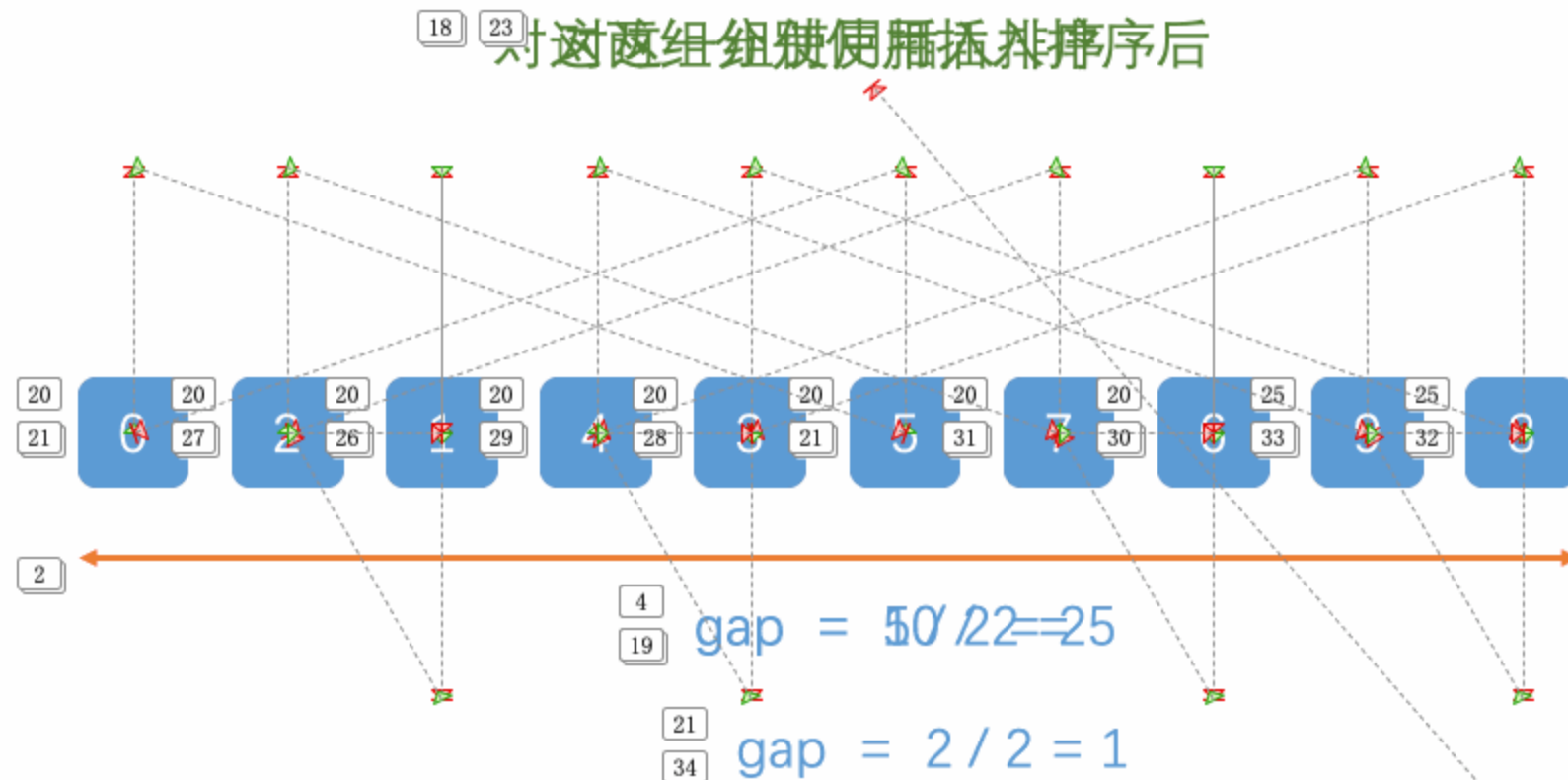
时间复杂度为 $O(n^2)$ 。

平均情况下（随机排列）：

{ 比较次数: $\sum_{i=2}^n \frac{i}{2} = \frac{(n+2)(n-1)}{4}$
移动次数: $\sum_{i=2}^n \frac{(i+1)}{2} = \frac{(n+4)(n-1)}{4}$

时间复杂度为 $O(n^2)$ 。

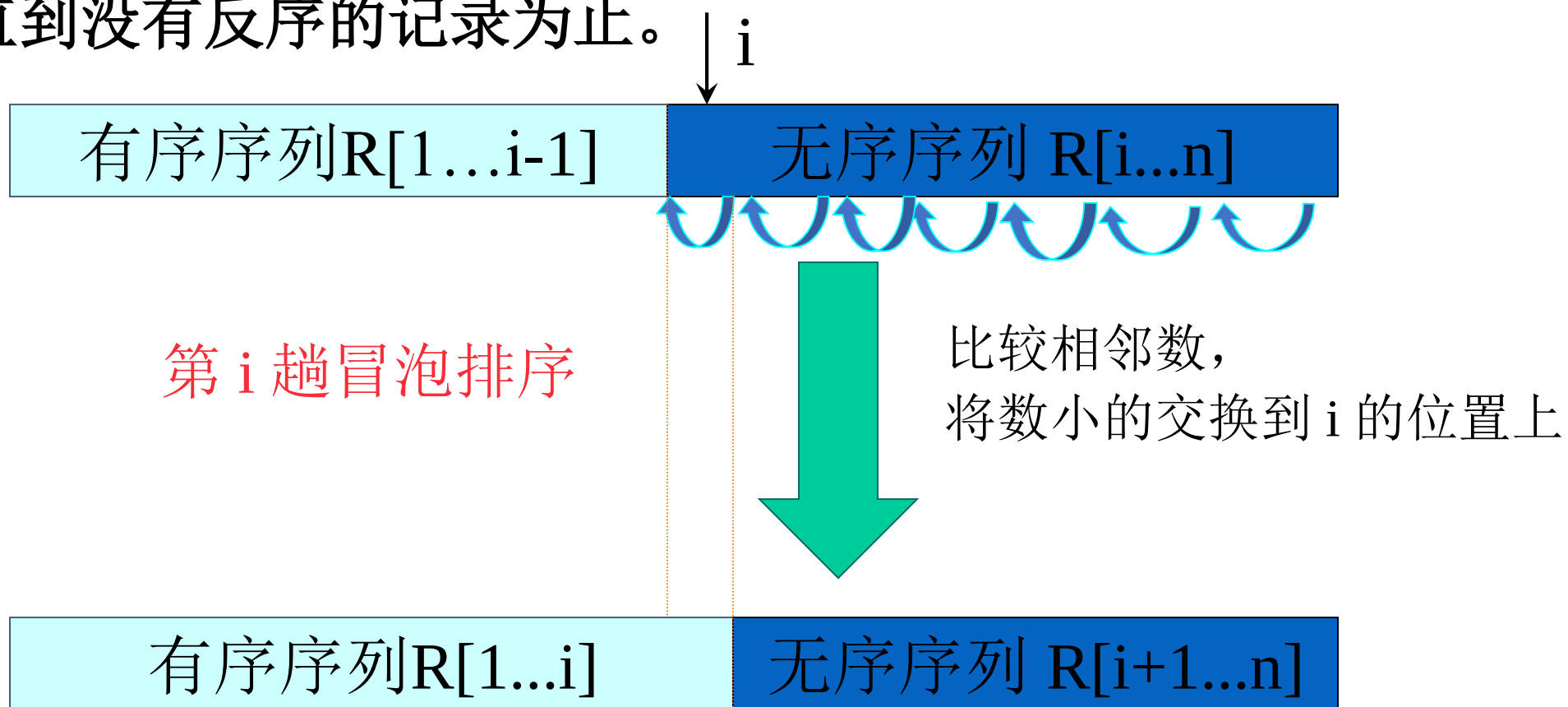
希尔冒泡排序过程示例



冒泡排序（小数上浮）

主要操作：交换

基本思想：从右向左，两两相邻数比较，如果反序则交换，直到没有反序的记录为止。



冒泡排序(小数上浮)过程示例



冒泡排序（小数上浮） 算法

```
void BubbleSort(int r[],int n)
{
    int i,j,exchange;
    for (i=1;i<=n-1;i++)
    {
        exchange=0;
        for (j=n;j>i;j--)
            if (r[j]<r[j-1])
            {
                r[j]  $\longleftrightarrow$  r[j-1];
                exchange=1;
            }
        if (exchange==0) return;
    }
}
```

```
void BubbleSort (int r[ ], int n)
{
    exchange=1;
    while (exchange)
    {
        bound= exchange;
        exchange=0;
        for (j=n; j>bound; j--)
            if (r[j-1]>r[j]) {
                r[j] $\longleftrightarrow$ r[j-1];
                exchange=j;
            }
    }
}
```

冒泡排序算法性能分析

最好情况（正序）：

比较次数： $n-1$
移动次数： 0

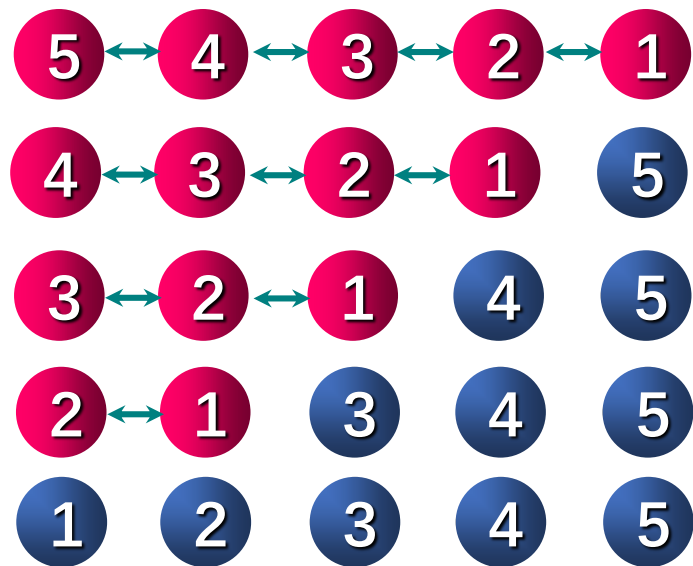
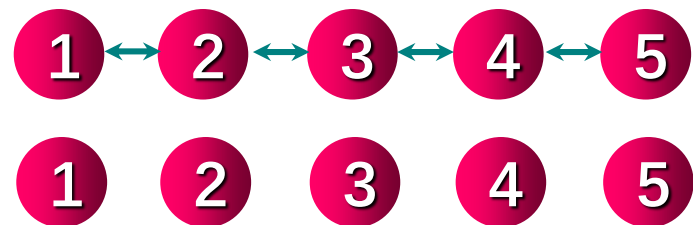
时间复杂度为 $O(n)$ ；

最坏情况（反序）：

比较次数： $\sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2}$
移动次数： $\sum_{i=1}^{n-1} 3(n-i) = \frac{3n(n-1)}{2}$

时间复杂度为 $O(n^2)$ 。

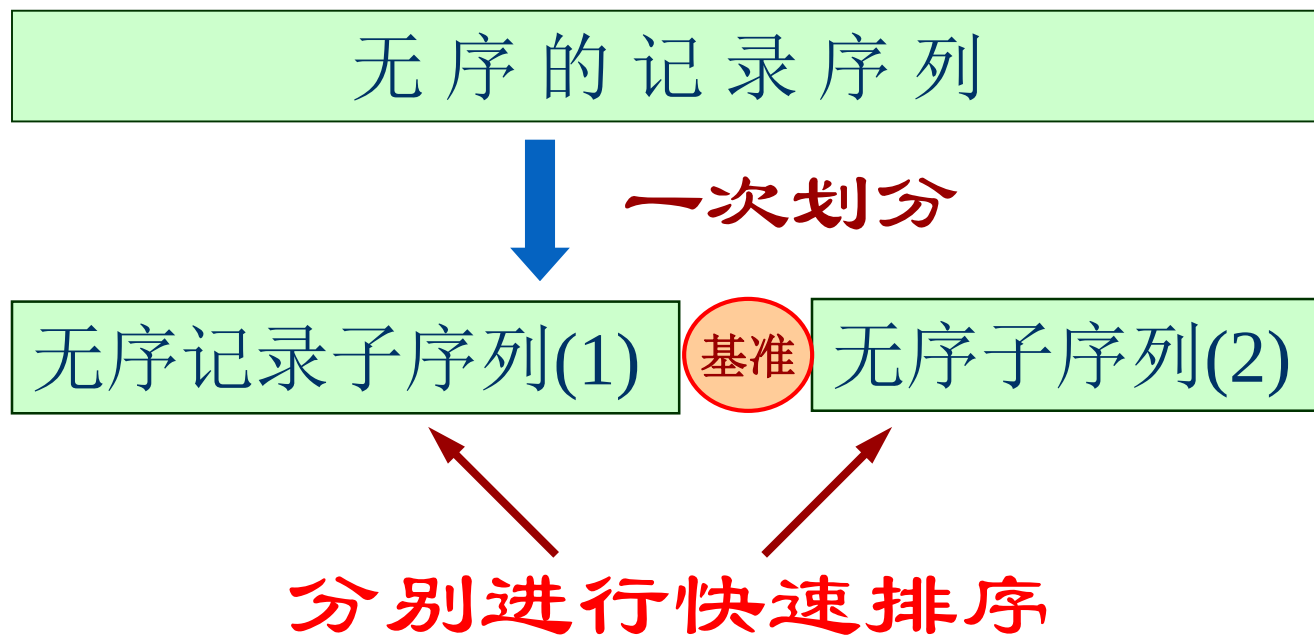
平均情况：时间复杂度为 $O(n^2)$ 。



快速排序过程示例（交换）

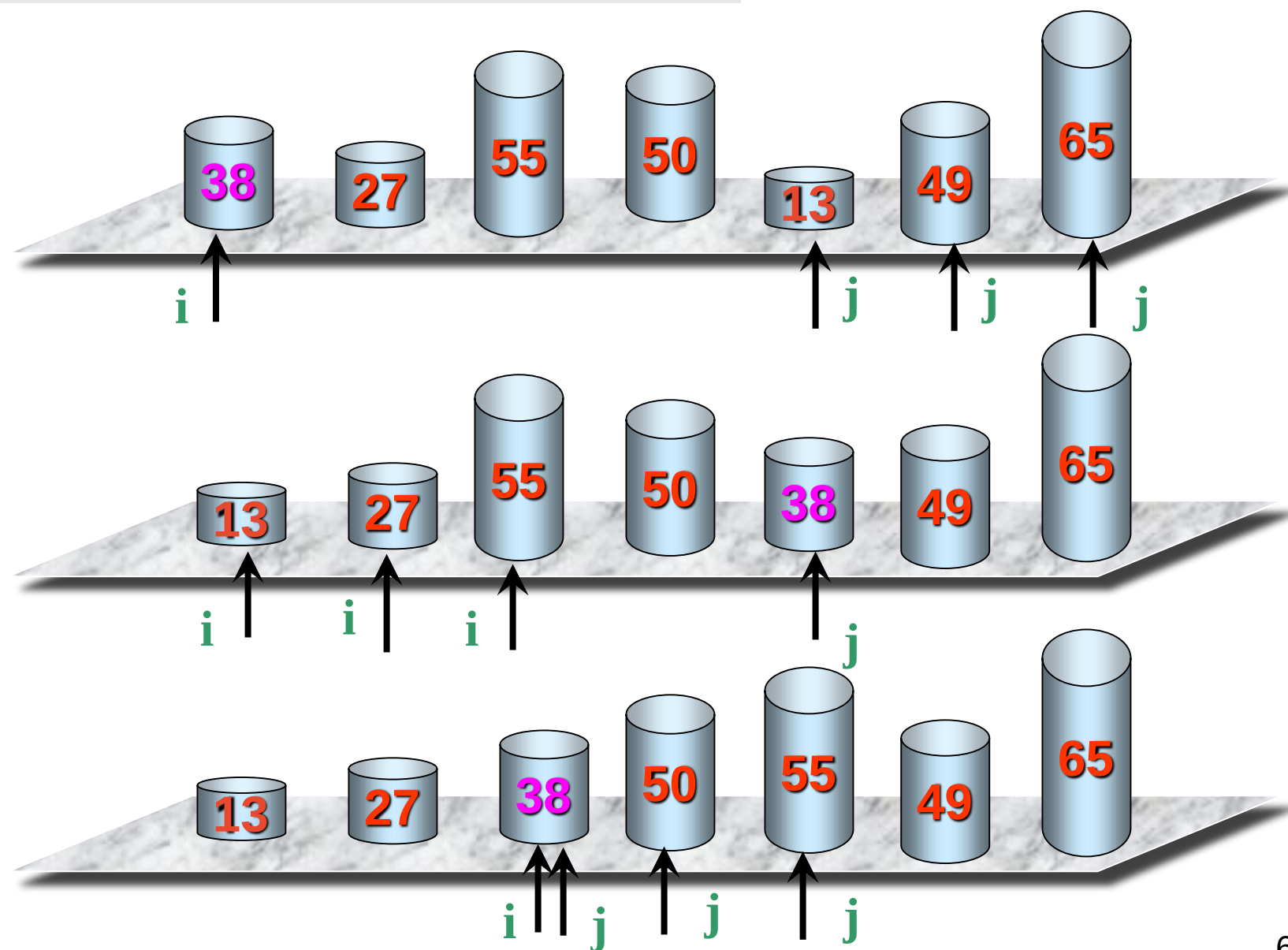
快速排序

首先，对无序的记录序列进行“一次划分”，
然后，分别对分割所得两个子序列“递归”进行快速排序。

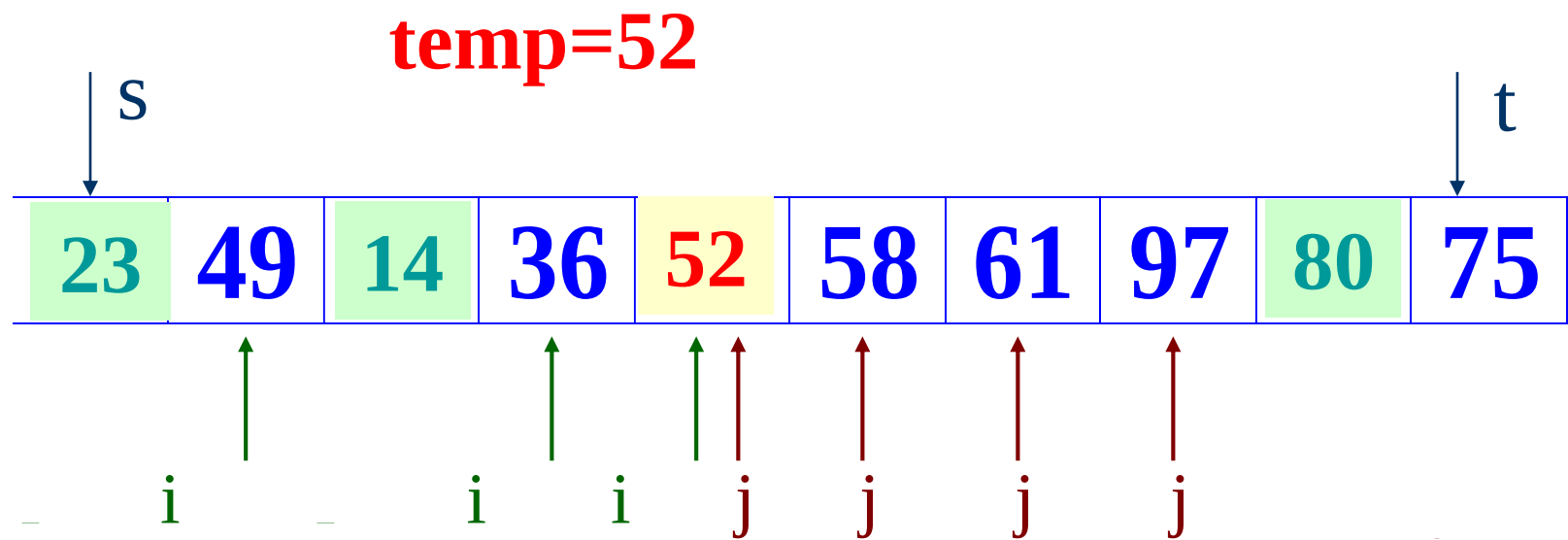


一般以数列的第一个或者最后一个作为为“基准”

快速排序过程示例(交换)



快速排序过程示例（赋值）



设 **temp = 52** 为基准

设置: **i**, **j**, **temp**

赋值

从**j**向左搜，找到第一个小于**temp**，和枢轴交换，**i++**

再从**i**向左搜，大，**j--**

重复，直到**i = j**为止。

快速排序算法性能分析

空间效率：快速排序是递归的，每层递归调用时的指针和参数均要用栈来存放，递归调用层次数与二叉树的深度一致。

存储开销在理想情况下为 $O(\log_2 n)$ ，即树的高度；
在最坏情况下，即二叉树是一个单链，为 $O(n)$ 。

时间效率：

理想情况下：每次划分，正好将分成两个等长的子序列，
则时间复杂度为 $O(n \log_2 n)$ ；

最坏情况下：快速排序每次划分，只得到一个子序列，
则退化为冒泡排序的过程，其时间复杂度最差，为 $O(n^2)$ 。

快速排序是通常被认为**平均性能最好**的算法。

快速排序是一个**不稳定**的排序算法。

快速排序一次划分算法

```
int Partition(int r[ ], int first, int end)
{
    i=first; j=end;           // 初始化
    temp = r[i];
    while (i<j)
    {
        while (i<j && r[i]<= r[j]) j--;
        if (i<j) {
            r[i]=r[j];    i++;
        }
        while (i<j && r[i]<= r[j]) i++;
        if (i<j) {
            r[j]=r[i];    j--;
        }
    }
    r[i] = temp;
    return i;
    // i为轴值记录的最终位置
}
```

```
void QuickSort(int r[ ], int first, int end)
{
    if (first<end) {
        pivot=Partition(r, first, end);
        //pivot是轴值在序列中的位置
        QuickSort(r, first, pivot-1);
        //递归地对左侧子序列进行快速排序
        QuickSort(r, pivot+1, end);
        //递归地对右侧子序列进行快速排序
    }
}
```

快速排序算法

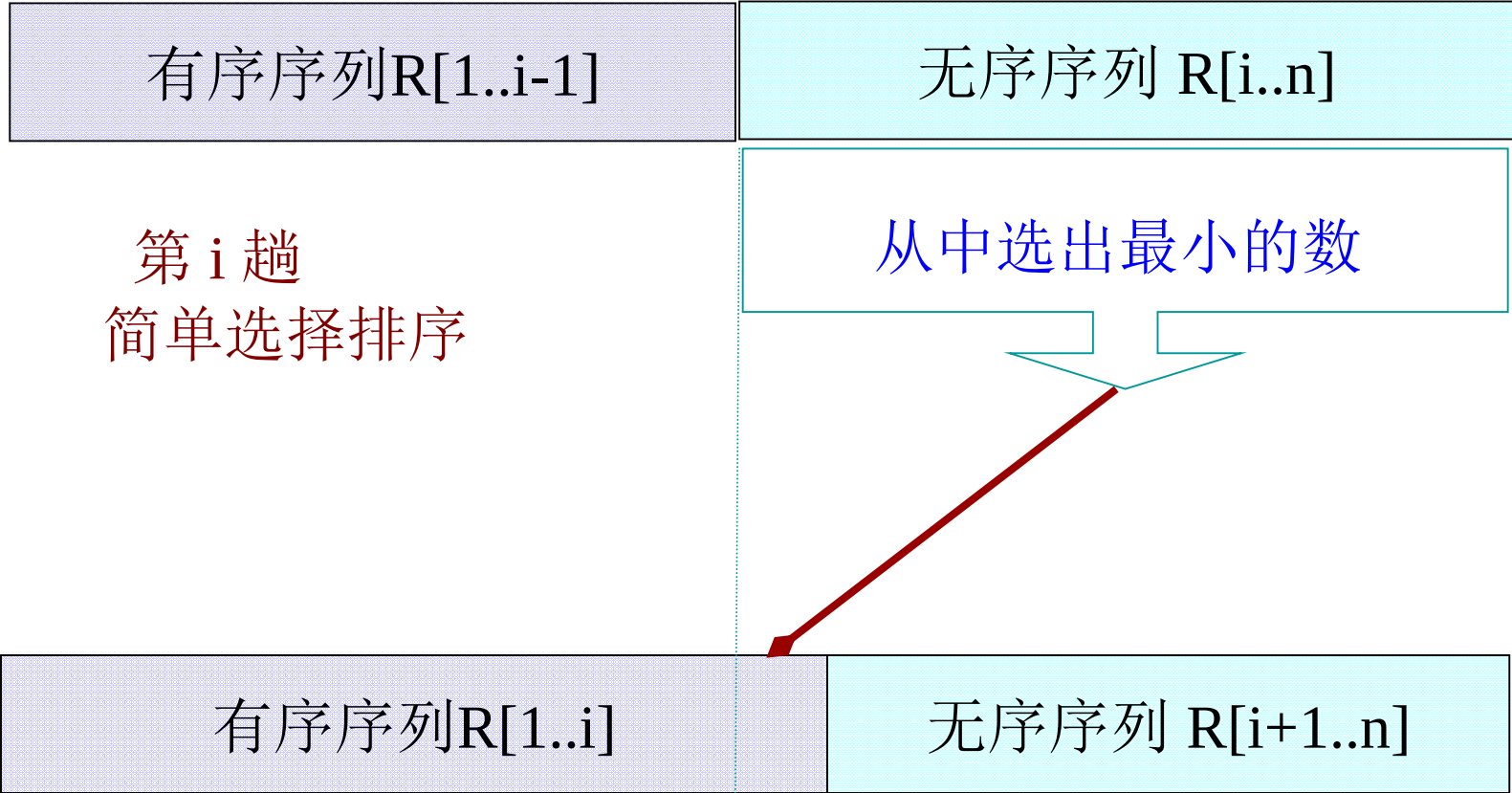
```
void QuickSort(int R[],int s,int t)
{
    int i=s,j=t;  int temp;
    if (s<t)
    {
        temp=R[s];           //用区间的第1个记录作为基准
        while (i!=j)
        {
            while (j>i && R[j]>temp)      j--;
            if (i<j)
            {
                R[i]=R[j]; i++;
            }
            while (i<j && R[i]<temp)      i++;
            if (i<j)
            {
                R[j]=R[i]; j--;
            }
        }
        R[i]=temp;
        QuickSort(R, s, i-1);           //对左区间递归排序
        QuickSort(R, i+1, t);           //对右区间递归排序
    }
}
```

划分

选择排序

主要操作：查找、选择

基本思想：每一趟在后面 $n-i+1$ 个中选最小的数，
作为有序序列的第 i 个记录



简单选择排序过程示例



简单选择排序过程示例

初始关键字	6	8	7	9	0	1	3	2	4	5
i=1	0	8	7	9	6	1	3	2	4	5
i=2	0	1	7	9	6	8	3	2	4	5
i=3	0	1	2	9	6	8	3	7	4	5
i=4	0	1	2	3	6	8	9	7	4	5
i=5	0	1	2	3	4	8	9	7	6	5
i=6	0	1	2	3	4	5	9	7	6	8
i=7	0	1	2	3	4	5	6	7	9	8
i=8	0	1	2	3	4	5	6	7	9	8
i=9	0	1	2	3	4	5	6	7	8	9

简单选择排序算法

```
void SelectSort(int r[],int n)
```

```
{  int i,j,k;  int temp;
```

```
    for (i=1;i<n;i++)           //做第i趟排序
```

```
    {  k=i;
```

```
        for (j=i+1;j<=n;j++)     //在[i..n-1]中选最小的r[k]
```

```
            if (r[j]<r[k])
```

```
                k=j;             //k记下的最小关键字所在的位置
```

```
        if (k!=i)               //交换r[i]和r[k]
```

```
        {  temp=r[i]; r[i]=r[k]; r[k]=temp; }
```

```
    }
```

```
}
```

简单选择排序性能分析

简单选择排序比较次数与关键字初始排序无关。

找第一个最小记录需进行 $n-1$ 次比较，

找第二个最小记录需要比较 $n-2$ 次，

找第 i 个最小记录需要进行 $n-i$ 次比较，

总的比较次数为： $(n-1) + (n-2) + \dots + (n-i) + \dots + 2 + 1$
 $= n(n-1)/2$

移动记录的次数：最小值为 0，最大值为 $3(n-1)$ 。

时间复杂度： $O(n^2)$ 辅助空间： $O(1)$

简单选择排序是**不稳定**的排序方法。

归并排序

- 将数组分成均匀两半；
- 递归解决每半部分；
- 把两部分合并起来组成一个新的排序。

A	L	G	O	R	I	T	H	M	S
---	---	---	---	---	---	---	---	---	---

A	L	G	O	R		I	T	H	M	S
---	---	---	---	---	--	---	---	---	---	---

divide

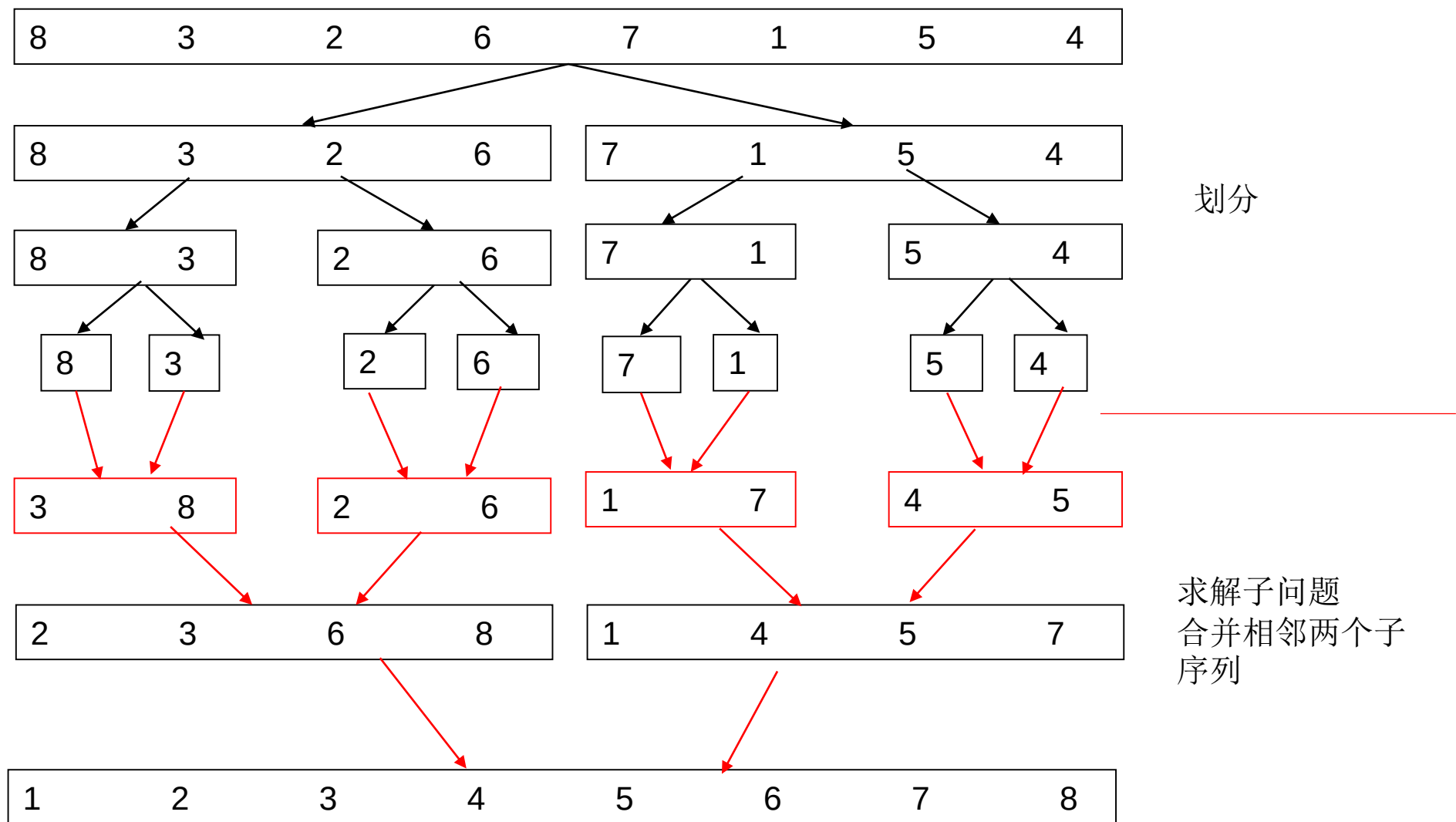
A	G	L	O	R		H	I	M	S	T
---	---	---	---	---	--	---	---	---	---	---

sort

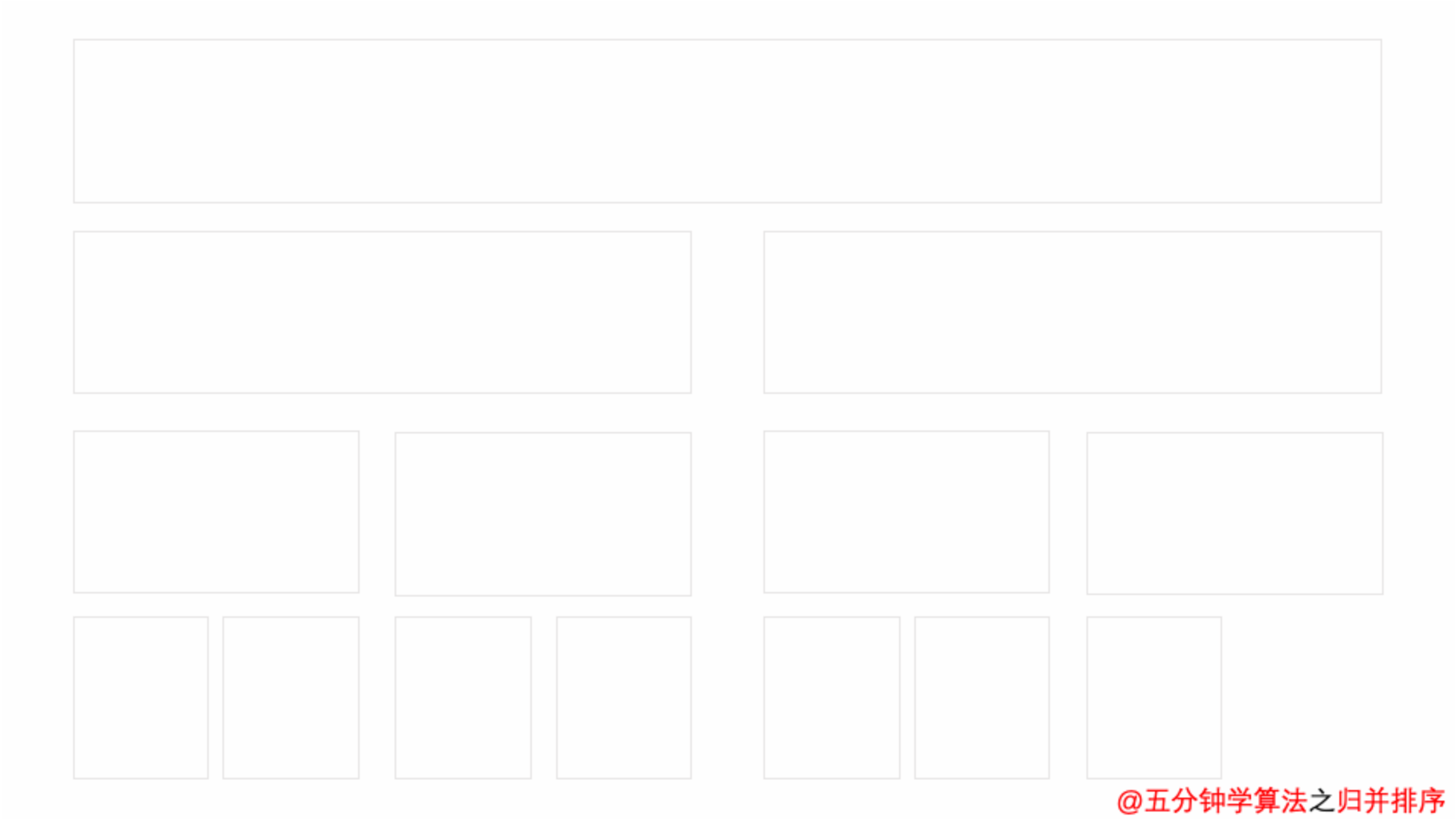
A	G	H	I	L	M	O	R	S	T
---	---	---	---	---	---	---	---	---	---

merge

归并排序



归并排序过程示例



归并排序算法

```
void MergeSort(int r[ ], int s, int t)
{
    int r1[1000], m;
    if (s==t) return;
    else {    m=(s+t)/2;
             MergeSort(r, s, m);
             MergeSort(r, m+1, t);
             Merge(r, r1, s, m, t);
             for(int i = s; i<=t; i++)    r[i] = r1[i];
    }
}

void Merge(int r[ ], int r1[ ], int s, int m, int t)
{
    i=s; j=m+1; k=s;
    while (i<=m && j<=t)
    {
        if (r[i]<=r[j])    r1[k++]=r[i++];
        else    r1[k++]=r[j++];
    }
    while (i<=m)    r1[k++]=r[i++];
    while (j<=t)    r1[k++]=r[j++];
}
```

归并排序算法性能分析

归并排序的合并步的时间复杂度为 $O(n)$,

所以, 归并排序算法存在如下递推式:

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n/2) + n & n > 1 \end{cases}$$

归并排序的时间复杂度是 $O(n\log_2 n)$ 。

归并排序在合并过程中需要与原始记录序列同样数量的存储空间,

因此, 其空间复杂度为 $O(n)$ 。

归并排序算法性能分析

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n/2) + n & n > 1 \end{cases}$$

当 $n=2^k$ 时, 可得

$$T(n) = 2(2T(n/2^2) + n/2) + n$$

$$= 4T(n/2^2) + 2n$$

$$= 4(2T(n/2^3) + n/4) + 2n$$

.....

$$= 2^k T(1) + kn$$

$$= n \log_2 n$$

如果 $2^k < n < 2^{k+1}$, 有 $T(n) \leq T(2^{k+1})$, 有 $T(n) = O(n \log_2 n)$ 。

排序算法	平均时间复杂度	最好情况	最坏情况	空间复杂度	排序方式	稳定性
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	In-place	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	In-place	不稳定
插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	In-place	稳定
希尔排序	$O(n \log n)$	$O(n \log^2 n)$	$O(n \log^2 n)$	$O(1)$	In-place	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Out-place	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	In-place	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	In-place	不稳定
计数排序	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Out-place	稳定
桶排序	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	Out-place	稳定
基数排序	$O(n \times k)$	$O(n \times k)$	$O(n \times k)$	$O(n + k)$	Out-place	稳定

堆排序



基数堆排序







